

Contents

Contents

1	图论	2	3.5	Generating function	11	6.10	凸包	22
1.1	二分图匹配	2	3.6	全在线卷积	11	6.11	上凸壳	23
1.1.1	二分图最大匹配 最小边覆盖	2	3.7	多项式复合	12	6.12	最小圆覆盖	23
1.1.2	二分图最大匹配 (bitset 优化)	2	3.8	Power Projection	12	6.13	最近点对	23
1.1.3	二分图最大匹配 (Hopcroft-Karp) & 最小边覆盖	2	3.9	常系数线性递推	12	6.14	凸包直径	23
1.1.4	二分图最小权匹配	2	3.9.1	Berlekamp Massey	12	6.15	切凸包	23
1.1.5	二分图最小权匹配2	3	3.9.2	递推式转分式	12	6.16	V 图	24
1.2	网络最大流 (dinic)	3	3.9.3	Bostan-Mori	13	6.17	Delaunay 三角剖分	24
1.3	网络最大流 (dinic) 2	4	3.9.4	远处区间值	13			
1.4	最小费用流	4	3.10	线性规划 (单纯形法)	13	7	geometry3d	25
1.5	一般图最大匹配 (带花树)	5	3.11	黄金三分	14	7.1	向量	25
1.6	最小树形图	5	3.12	特征多项式	14	7.2	平面	25
1.7	强连通分量 (kosaraju)	6	3.13	det(A + B x)	14	7.3	直线	25
1.8	欧拉回路/欧拉路径	6				7.4	凸包	25
1.9	支配树	7	4	字符串	15	8	Misc	26
1.10	Tree And Graph	7	4.1	后缀自动机 SAM	15	8.1	Pragma	26
1.10.1	树的计数 Prufer 序列	7	4.2	回文自动机 PAM	15	8.2	Barrett	26
1.10.2	有根树的计数	7	4.3	回文自动机 PAM dp	15	8.3	快速整除判定	26
1.10.3	无根树的计数	7	4.4	AC 自动机	16	8.4	Montgomery	26
1.10.4	生成树计数 Kirchhoff's Matrix-Tree Thoerem	7	4.5	SA	16	8.5	Pragma	26
1.10.5	有向图欧拉回路计数 BEST Thoerem	7	4.6	Z algorithm	16	8.6	LCS	26
1.10.6	Tutte Matrix	7	4.7	Manacher	16	8.7	日期公式	27
1.10.7	Edmonds Matrix	8	4.8	KMP	17	8.8	Xorshift	27
1.10.8	Erdős-Gallai theorem	8	4.9	最小表示法	17	8.9	distributions	27
			4.10	Lyndon 分解	17	8.10	python	27
2	数论	8				8.11	浮点数精度	27
2.1	取模还原分数	8	5	数据结构	17	9	配置	27
2.2	扩展欧几里得	8	5.1	区间加区间求和和树状数组	17	9.1	vimrc	27
2.3	万能欧几里得	8	5.2	zkw 线段树	17	9.2	bashrc	27
2.4	floor-sum	8	5.3	静态区间半群	18	9.3	对拍	27
2.5	Min of Mod of Linear	8	5.4	Link Cut Tree	18	9.4	编译参数	27
2.6	Stern-Brocot Tree 二分	9	5.5	压位 Trie	18	9.5	随机素数	28
2.7	中国剩余定理	9	5.6	pbds tree	19	9.6	常数表	28
2.8	Miller-Rabin	9	6	geometry	19	10	注意事项	28
2.9	Pollard-rho	9	6.1	向量	19	10.1	测试项目	28
2.10	二次剩余	9	6.2	三角形各心	20	10.2	bugs	28
3	Math	10	6.3	直线半平面	20	11	tables	29
3.1	拉格朗日反演	10	6.4	半平面交	20	11.1	导数积分	29
3.2	分拆数 五边形数	10	6.5	线段	20			
3.3	Fast Fourier Transform	10	6.6	简单多边形	21			
3.4	Number Theoretic Transform	10	6.7	简单多边形三角剖分	21			
			6.8	线段 in 多边形	21			
			6.9	图形求交	22			

1 图论

1.1 二分图匹配

1.1.1 二分图最大匹配 | 最小边覆盖

左到右连单向边，时间复杂度 $O(M|match|)$ 。

```

1 std::vector<int> edge[N];
2 std::vector<int> match(int nl, int nr) {
3     std::vector<int> vis(nr + 1), match(nr + 1), ret(nl + 1);
4     auto dfs = [&](auto dfs, int x) -> int {
5         for(int y : edge[x]) if(!vis[y])
6             if(vis[y] = 1, !match[y] || dfs(dfs, match[y]))
7                 return match[y] = x, 1;
8         return 0;
9     };
10    for(int i = 1; i <= nl; ++i) if(dfs(dfs, i))
11        memset(vis.data(), 0, vis.size() << 2);
12    for(int i = 1; i <= nr; ++i) if(int t = match[i]) ret[t] = i;
13    return ret;
14 }
15 std::array<std::vector<int>, 3> minedgeconver(int nl, int nr) {
16     std::vector<int> vis(nr + 1), match(nr + 1), ret(nl + 1);
17     auto dfs = [&](auto dfs, int x) -> int {
18         for(int y : edge[x]) if(!vis[y])
19             if(vis[y] = 1, !match[y] || dfs(dfs, match[y]))
20                 return match[y] = x, 1;
21         return 0;
22     };
23    for(int i = 1; i <= nl; ++i) if(dfs(dfs, i))
24        memset(vis.data(), 0, vis.size() << 2);
25    for(int i = 1; i <= nr; ++i) if(int t = match[i]) ret[t] = i;
26    for(int i = 1; i <= nl; ++i) if(!ret[i]) dfs(dfs, i);
27    std::vector<int> le, ri;
28    for(int i = 1; i <= nl; ++i) if(ret[i] && !vis[ret[i]]) le.push_back(i);
29    for(int i = 1; i <= nr; ++i) if(vis[i]) ri.push_back(i);
30    return {le, ri, ret};
31 }

```

1.1.2 二分图最大匹配 (bitset 优化)

左到右连单向边，时间复杂度 $O(n^2/w |match|)$ 。

```

1 const int N = 505;
2 using set = std::bitset<N>;
3 set edge[N];
4 std::vector<int> match(int nl, int nr) {
5     set unvis; unvis.set();
6     std::vector<int> match(nr + 1), ret(nl + 1);
7     auto dfs = [&](auto dfs, int x) {
8         for(set z = edge[x];;) {
9             z &= unvis;
10            int y = z._Find_first();
11            if(y == N) return 0;

```

```

12         if(unvis.reset(y), !match[y] || dfs(dfs, match[y])) {
13             return match[y] = x, 1;
14         }
15     };
16 };
17 for(int i = 1; i <= nl; ++i)
18     if(dfs(dfs, i)) unvis.set();
19 for(int i = 1; i <= nr; ++i) ret[match[i]] = i;
20 return ret[0] = 0, ret;
21 }

```

1.1.3 二分图最大匹配 (Hopcroft-Karp) & 最小边覆盖

左到右单向边，时间复杂度 $O(M\sqrt{|match|})$

```

1 std::vector<int> e[N];
2 std::vector<int> matchl, matchr, a, p;
3 std::vector<int> match(int nl, int nr) {
4     matchl.assign(nl + 1, 0), matchr.assign(nr + 1, 0);
5     for(;;) {
6         a.assign(nl + 1, 0), p.assign(nl + 1, 0);
7         static std::queue<int> Q;
8         for(int i = 1; i <= nl; ++i)
9             if(!matchl[i]) a[i] = p[i] = i, Q.push(i);
10        int succ = 0;
11        for(; Q.size(); ) {
12            int x = Q.front(); Q.pop();
13            if(matchl[a[x]]) continue;
14            for(int y : e[x]) {
15                if(!matchr[y]) {
16                    for(succ = 1; y; x = p[x])
17                        matchr[y] = x, std::swap(matchl[x], y);
18                    break;
19                }
20                if(!p[matchr[y]])
21                    Q.push(y = matchr[y]), p[y] = x, a[y] = a[x];
22            }
23        }
24        if(!succ) break;
25    }
26    return matchl;
27 }
28 std::array<std::vector<int>, 3> minedgecover(int nl, int nr) {
29     match(nl, nr);
30     std::vector<int> l, r;
31     for(int i = 1; i <= nl; ++i) if(!a[i]) l.push_back(i);
32     for(int i = 1; i <= nr; ++i) if(a[matchr[i]]) r.push_back(i);
33     return {l, r, matchl};
34 }

```

1.1.4 二分图最小权匹配

返回左边每个点匹配的右边点的编号，左边的点不多于右边的点（必要时补点）。

```

1 ll e[N][N];
2 std::vector<int> KM(int n, int m) {
3     std::vector<ll> l(n + 1), r(m + 1);
4     std::vector<int> p(m + 1), ans(n + 1);
5     for(int i = 1; i <= n; ++i) {
6         std::vector<ll> d(m + 1, 1e18);
7         std::vector<int> pre(m + 1), done(m + 1);
8         int x, v, u = 0;
9         for(p[0] = i; x = p[u]; u = v) {
10             done[u] = 1;
11             ll min = 1e18;
12             for(int j = 1; j <= m; ++j) if(!done[j]) {
13                 auto w = e[x][j] - l[x] - r[j];
14                 if(w < d[j]) d[j] = w, pre[j] = u;
15                 if(d[j] < min) min = d[j], v = j;
16             }
17             for(int j = 0; j <= m; ++j) {
18                 if(done[j]) l[p[j]] += min, r[j] -= min;
19                 else d[j] -= min;
20             }
21         }
22         for(int v; u = v) {
23             v = pre[u], p[u] = p[v];
24         }
25     }
26     for(int j = 1; j <= m; ++j) ans[p[j]] = j;
27     return ans; // 权值和 : -r[0]
28 }

```

1.1.5 二分图最小权匹配2

最小权匹配。返回代价和、右边每个点匹配的左边点的编号、对应的边权。过程中能计算出匹配为 $0 \cdots |match|$ 的最优解。

时间复杂度 $O(|match|(L^2 + |E|))$ 。

```

1 std::vector<std::pair<int, ll>> e[N];
2 bool down(ll &x, ll y) { return x > y ? x = y, 1 : 0; }
3 auto KM(int n, int m) {
4     std::vector<ll> h(n + 1), rv(m + 1);
5     std::vector<int> R(m + 1);
6     ll cost = 0;
7     for(;;) {
8         std::vector<ll> d(n + 1);
9         std::vector<int> vis(n + 1), p0(n + 1), p1(m + 1);
10        for(int i = 0; i <= m; ++i) d[R[i]] = inf;
11        for(int x = 0; vis[x] ^= 1; ) {
12            ll min = 1e18;
13            for(auto [j, w] : e[x]) if(down(d[R[j]], d[x] + w - rv[j])) p0[R[j]]
14            = j, p1[j] = x;
15            for(int j = 1; j <= n; ++j) if(!vis[j] && down(min, d[j] - h[j])) x =
16            j;
17        }
18        if(d[0] >= inf / 2) break; // or inf/2==0

```

```

17         for(int r = p0[0]; r; r = p0[R[r]]) {
18             rv[r] += d[R[r]] - d[p1[r]];
19             R[r] = p1[r];
20         }
21         cost += d[0], h = d;
22     }
23     return std::tuple(cost, R, rv);
24 }

```

1.2 网络最大流 (dinic)

```

1 // S 编号最小, T 最大, 或者改一下清空
2 struct Dinic {
3     struct T {
4         int to, nxt, v;
5     } e[N << 3];
6     int h[N], head[N], num = 1;
7     void link(int x, int y, int v) {
8         e[++num] = {y, h[x], v}, h[x] = num;
9         e[++num] = {x, h[y], 0}, h[y] = num; // !!!
10    }
11    int dis[N];
12    bool bfs(int s, int t) {
13        std::queue<int> Q;
14        for(int i = s; i <= t; ++i) dis[i] = -1, head[i] = h[i]; //如果编号不是
15        [S,T], 只要改这里
16        for(Q.push(t), dis[t] = 0; !Q.empty(); ) {
17            int x = Q.front(); Q.pop();
18            for(int i = h[x]; i; i = e[i].nxt) if(e[i].v && dis[e[i].to] < 0)
19            {
20                dis[e[i].to] = dis[x] + 1, Q.push(e[i].to);
21            }
22        }
23        return dis[s] >= 0;
24    }
25    int dfs(int s, int t, int lim) {
26        if(s == t || !lim) return lim;
27        int ans = 0, mn;
28        for(int &i = head[s]; i; i = e[i].nxt) {
29            if(dis[e[i].to] + 1 == dis[s] && (mn = dfs(e[i].to, t, std::min(lim,
30            e[i].v)))) {
31                e[i].v -= mn, e[i ^ 1].v += mn;
32                ans += mn, lim -= mn;
33                if(!lim) break;
34            }
35        }
36        return ans;
37    }
38    int flow(int s, int t) {
39        int ans = 0;
40        for(;;bfs(s, t);) ans += dfs(s, t, 1e9);
41        return ans;
42    }
43 } G;

```

1.3 网络最大流 (dinic) 2

这是 vector 版本, 但不知道为什么很多情况不会快。至少清空好搞, 没有经过严谨测试。

```

1 struct maxflow {
2     int n;
3     std::vector<std::vector<std::array<int, 2>>> e;
4     std::vector<ll> f;
5     maxflow(int s) : n(s + 1), e(n) {}
6     void add(int x, int y, int v) {
7         e[x].push_back({y, (int)f.size()});
8         f.push_back(v);
9         e[y].push_back({x, (int)f.size()});
10        f.push_back(0);
11    }
12    ll flow(int s, int t, ll max = 1e18) {
13        ll ans = 0;
14        std::vector<int> dis(n), h(n);
15        for(; ans < max; ) {
16            for(int i = 0; i < n; ++i) dis[i] = -1, h[i] = e[i].size() - 1;
17            std::queue<int> q;
18            for(q.push(t), dis[t] = 0; q.size(); ) {
19                int x = q.front(); q.pop();
20                for(auto [y, id] : e[x]) if(dis[y] < 0 && f[id ^ 1]) {
21                    dis[y] = dis[x] + 1, q.push(y);
22                    if(y == s) break;
23                }
24            }
25            if(dis[s] < 0) break;
26            auto dfs = [&](auto dfs, int s, ll l) {
27                if(s == t) return l;
28                ll ans = 0;
29                for(int & i = h[s]; i >= 0; --i) {
30                    auto [y, id] = e[s][i];
31                    if(dis[y] + 1 != dis[s] || !f[id]) continue;
32                    ll w = dfs(dfs, y, std::min(l - ans, f[id]));
33                    f[id] -= w, f[id ^ 1] += w;
34                    ans += w;
35                    if(ans == l) return l;
36                }
37                dis[s] = -1;
38                return ans;
39            };
40            ans += dfs(dfs, s, max - ans);
41        }
42        return ans;
43    }
44    auto getedges(int x) {
45        std::vector<std::pair<int, ll>> o;
46        for(auto [y, id] : e[x]) o.emplace_back(y, f[id]);
47        return o;
48    }
49 };

```

1.4 最小费用流

```

1 // S 编号最小, T 最大, 或者改一下清空
2 namespace mcmf {
3     using pr = std::pair<ll, int>;
4     const int N = 10005, M = 1e6 + 10;
5     struct edge {
6         int to, nxt, v, f;
7     } e[M << 1];
8     int h[N], num = 1;
9     void link(int x, int y, int v, int f) {
10        e[++num] = {y, h[x], v, f}, h[x] = num;
11        e[++num] = {x, h[y], 0, -f}, h[y] = num;
12    }
13    ll d[N], dis[N];
14    int vis[N], fr[N];
15    bool spfa(int s, int t) {
16        std::queue<int> Q;
17        std::fill(d + s, d + t + 1, 1e18); // CHECK
18        for(d[s] = 0, Q.push(s); !Q.empty(); ) {
19            int x = Q.front(); Q.pop(); vis[x] = 0;
20            for(int i = h[x]; i; i = e[i].nxt)
21                if(e[i].v && d[e[i].to] > d[x] + e[i].f) {
22                    d[e[i].to] = d[x] + e[i].f;
23                    fr[e[i].to] = i;
24                    if(!vis[e[i].to]) vis[e[i].to] = 1, Q.push(e[i].to);
25                }
26        }
27        return d[t] < 1e17;
28    }
29    bool dijkstra(int s, int t) { // 正常题目不需要 dijk
30        std::priority_queue<pr, std::vector<pr>, std::greater<pr>> Q;
31        for(int i = s; i <= t; ++i) dis[i] = d[i], d[i] = 1e18, vis[i] = fr[i] = 0; // CHECK
32        for(Q.emplace(d[s] = 0, s); !Q.empty(); ) {
33            int x = Q.top().second; Q.pop();
34            if(vis[x]) continue;
35            vis[x] = 1;
36            for(int i = h[x]; i; i = e[i].nxt) {
37                const ll v = e[i].f + dis[x] - dis[e[i].to];
38                if(e[i].v && d[e[i].to] > d[x] + v) {
39                    fr[e[i].to] = i;
40                    Q.emplace(d[e[i].to] = d[x] + v, e[i].to);
41                }
42            }
43        }
44        for(int i = s; i <= t; ++i) d[i] += dis[i]; // CHECK
45        return d[t] < 1e17;
46    }
47    std::pair<ll, ll> EK(int s, int t) {
48        spfa(s, t); // 如果初始有负权且要 dijk
49        ll f = 0, c = 0;
50        for(; dijkstra(s, t); ) { // 正常可以用 spfa
51            ll fl = 1e18;

```

```

52 | | | for(int i = fr[t]; i = fr[e[i ^ 1].to]) fl = std::min<ll>(e[i].v,
    | | | fl);
53 | | | for(int i = fr[t]; i = fr[e[i ^ 1].to]) e[i].v -= fl, e[i ^ 1].v +=
    | | | fl;
54 | | | f += fl, c += fl * d[t];
55 | | }
56 | | return std::make_pair(f, c);
57 | }
58 | }
59 // in flow problems with lower bounds (or with negative cycles), flow the
    // negative edges first
60 // after the first round, revert the auxiliary edges

```

1.5 一般图最大匹配 (带花树)

edge 里存双向边。运行结束后 match 存放每个点匹配的点的编号，如果没有匹配则为 0。

```

1 int n;
2 std::vector<int> edge[N];
3 int pre[N], match[N], fa[N], vis[N], t[N];
4 int find(int x) {
5 | return fa[x] = x ? x : fa[x] = find(fa[x]);
6 }
7 void aug(int s) {
8 | std::queue<int> Q;
9 | for(int i = 1; i <= n; ++i) {
10 | | pre[i] = vis[i] = t[i] = 0, fa[i] = i;
11 | }
12 | auto lca = [&](int x, int y) {
13 | | static int tick = 0; ++tick;
14 | | for(;; std::swap(x = pre[match[x]], y)) {
15 | | | if(vis[x = find(x)] == tick) return x;
16 | | | vis[x] = x ? tick : 0;
17 | | }
18 | };
19 | auto blossom = [&](int x, int y, int z) {
20 | | for(;; find(x) != z; x = pre[y]) {
21 | | | pre[x] = y, y = match[x], fa[x] = fa[y] = z;
22 | | | if(t[y] == 2) Q.push(y), t[y] = 1;
23 | | }
24 | };
25 | Q.push(s), t[s] = 1;
26 | for(;; Q.size(); ) {
27 | | int x = Q.front(); Q.pop();
28 | | for(int y : edge[x]) {
29 | | | if(t[y] == 2) continue;
30 | | | if(t[y] == 1) {
31 | | | | int z = lca(x, y);
32 | | | | blossom(x, y, z);
33 | | | | blossom(y, x, z);
34 | | | } else if (int p = match[y]) {
35 | | | | pre[y] = x, t[y] = 2, t[p] = 1;
36 | | | | Q.push(p);
37 | | | } else {

```

```

38 | | | int z;
39 | | | for(pre[y] = x; y; y = z) {
40 | | | | x = pre[y], z = match[x];
41 | | | | match[x] = y, match[y] = x;
42 | | | }
43 | | | return ;
44 | | }
45 | }
46 | }
47 | }
48 void run() {
49 | for(int i = 1; i <= n; ++i) if(!match[i]) {
50 | | aug(i);
51 | }
52 | }

```

1.6 最小树形图

用 DMST::link 添加边，然后调用 DMST::solve(root) 得到最小树形图边的集合，如果没有合法解会返回空集 (注意 $n = 1$)。

```

1 namespace DMST {
2 | struct edge {
3 | | int u, v, id; ll w;
4 | | bool operator < (const edge & y) const {
5 | | | return w < y.w;
6 | | }
7 | } ent[N], val[M];
8 | int ls[M], rs[M], size[M], cc; ll tag[M];
9 | int fs[N], fw[N], rt[N];
10 | void put(int x, ll v) {
11 | | if(x) val[x].w += v, tag[x] += v;
12 | }
13 | void pushdown(int x) {
14 | | put(ls[x], tag[x]);
15 | | put(rs[x], tag[x]);
16 | | tag[x] = 0;
17 | }
18 | int merge(int x, int y) {
19 | | if(!x || !y) return x | y;
20 | | if(val[y] < val[x]) std::swap(x, y);
21 | | pushdown(x), rs[x] = merge(rs[x], y);
22 | | if(size[rs[x]] > size[ls[x]]) {
23 | | | std::swap(ls[x], rs[x]);
24 | | }
25 | | size[x] += size[y];
26 | | return x;
27 | }
28 | void ins(int & x, const edge & z) {
29 | | val[++cc] = z, size[cc] = 1;
30 | | x = merge(x, cc);
31 | }
32 | void pop(int & x) { x = merge(ls[x], rs[x]); }
33 | edge top(int x) { return val[x]; }

```

```

34 | int find(int x, int * anc) {
35 |     return anc[x] = x ? x : anc[x] = find(anc[x], anc);
36 | }
37 | void link(int u, int v, int w, int id) {
38 |     ins(rt[v], {u, v, id, w});
39 | }
40 | int pa[N * 2], tval[N * 2], up[N * 2], end_edge[M], cmt, baned[M];
41 | std::vector<int> solve(int r) {
42 |     std::queue<int> roots;
43 |     for(int i = 1; i <= n; ++i) {
44 |         fs[i] = fw[i] = i, tval[i] = ++ cmt;
45 |         if(i != r) roots.push(i);
46 |     }
47 |     std::vector<edge> H;
48 |     std::vector<int> ret;
49 |     for(; !roots.empty(); ) {
50 |         int k = roots.front(); roots.pop();
51 |         if(!rt[k]) return ret;
52 |         edge e = top(rt[k]); pop(rt[k]);
53 |         int i = e.u, j = e.v;
54 |         if(find(i, fs) == k) roots.push(k);
55 |         else {
56 |             H.push_back(e); end_edge[e.id] = tval[k];
57 |             if(find(i, fw) != find(j, fw)) {
58 |                 fw[find(j, fw)] = i;
59 |                 ent[k] = e;
60 |             } else {
61 |                 pa[tval[k]] = ++ cmt, up[tval[k]] = e.id;
62 |                 put(rt[k], -e.w);
63 |                 for(; (e = ent[find(e.u, fs)].u); ) {
64 |                     int p = find(e.v, fs);
65 |                     pa[tval[p]] = cmt;
66 |                     up[tval[p]] = e.id;
67 |                     put(rt[p], -e.w);
68 |                     rt[k] = merge(rt[k], rt[p]);
69 |                     fs[p] = k;
70 |                 }
71 |                 tval[k] = cmt;
72 |                 roots.push(k);
73 |             }
74 |         }
75 |     }
76 |     reverse(H.begin(), H.end());
77 |     for(edge i : H) if(!baned[i.id]) {
78 |         ret.push_back(i.id);
79 |         for(int j = i.v; j != end_edge[i.id]; j = pa[j]) ++ baned[up[j]];
80 |     }
81 |     sort(ret.begin(), ret.end());
82 |     return ret;
83 | }
84 | }

```

1.7 强连通分量 (kosaraju)

e 存有向边, re 里存反向边。用 solve 得到所有强连通分量。时间复杂度 $O(\frac{n^2}{w})$ 。

```

1 | using set = std::bitset<N>;
2 | // re 是反向边, 需要连好
3 | set e[N], re[N], vis;
4 | std::vector<int> sta;
5 | void dfs0(int x, set * e) {
6 |     vis.reset(x);
7 |     for(;;) {
8 |         int go = (e[x] & vis)._Find_first();
9 |         if(go == N) break;
10 |         dfs0(go, e);
11 |     }
12 |     sta.push_back(x);
13 | }
14 | std::vector<std::vector<int>> solve() {
15 |     vis.set();
16 |     for(int i = 1; i <= n; ++i) if(vis.test(i)) dfs0(i, e);
17 |     vis.set();
18 |     auto s = sta;
19 |     std::vector<std::vector<int>> ret;
20 |     for(int i = n - 1; i >= 0; --i) if(vis.test(s[i])) {
21 |         sta.clear(), dfs0(s[i], re), ret.push_back(sta);
22 |     }
23 |     return ret;
24 | }

```

1.8 欧拉回路/欧拉路径

如果是无向图, 两个方向都连边, id 相同。不同的边 id 不同且在 $[1, m]$ 里。

src 是起点, 如果是欧拉路径, 找一个度数为奇数 (无向图)/出度大于入度 (有向图) 的点作为 src, 如果是回路, 选一个度数非零的点作为 src。

如果有解, 求出路径是 s, 那么返回 $[(s_1 = src, -1), (s_2, id(s_1 \rightarrow s_2)), \dots, (s_n, id(s_{n-1} \rightarrow s_n))]$, 如果是回路, 那么 $s_n = s_1 = src$ 。无解则返回空集。

```

1 | std::vector<pr> e[N]; // (dest, id)
2 | std::vector<pr> eulerwalk(int n, int m, int src) {
3 |     std::vector<int> d(n + 1), it(n + 1), vis(m + 1);
4 |     std::vector<pr> ret, s = {pr(src, -1)};
5 |     // ++ d[src]; // 如果需要欧拉路径, 加上这行
6 |     for(; s.size(); ) {
7 |         auto [x, id] = s.back();
8 |         int & i = it[x];
9 |         if(i == (int) e[x].size()) {
10 |             ret.push_back(s.back()), s.pop_back();
11 |             continue;
12 |         }
13 |         auto [y, o] = e[x][i++];
14 |         if(!vis[o]) {
15 |             -- d[x], ++ d[y];

```

```

16 | | | vis[o] = 1, s.emplace_back(y, o);
17 | | }
18 | }
19 | for(int x : d) if(x < 0 || size(ret) != m + 1) return {};
20 | return std::vector(ret.rbegin(), ret.rend());
21 | }

```

1.9 支配树

```

1 | std::vector<int> e[N];
2 | struct domtree {
3 | | std::vector<int> dfn, p, sdom, dom, anc, min, pa;
4 | | std::vector<std::vector<int>> b, re;
5 | | int cc;
6 | | void dfs0(int x) {
7 | | | dfn[x] = ++cc, p[cc] = x;
8 | | | for(int y : e[x]) {
9 | | | | if(!dfn[y]) dfs0(y), pa[dfn[y]] = dfn[x];
10 | | | | re[dfn[y]].push_back(dfn[x]);
11 | | | }
12 | | }
13 | | int find(int x) {
14 | | | int t = anc[x];
15 | | | if(anc[t] == t) return t;
16 | | | anc[x] = find(t);
17 | | | if(sdom[min[t]] < sdom[min[x]]) min[x] = min[t];
18 | | | return anc[x];
19 | | }
20 | | int get(int x) { return find(x), min[x]; }
21 | domtree() : b(n + 1), re(n + 1), cc(0) {}
22 | auto getdom(int s) {
23 | | pa = dfn = p = dom = anc = std::vector<int>(n + 1);
24 | | dfs0(s);
25 | | for(int i = 1; i <= cc; ++i) anc[i] = i;
26 | | dom = sdom = min = anc;
27 | | for(int i = cc; i-- > 0) {
28 | | | for(int w : re[i]) sdom[i] = std::min(sdom[i], sdom[get(w)]);
29 | | | if(i > 1) b[sdom[i]].push_back(i);
30 | | | for(int w : b[i]) {
31 | | | | int v = get(w);
32 | | | | dom[w] = sdom[v] == sdom[w] ? sdom[w] : v;
33 | | | }
34 | | | anc[i] = pa[i];
35 | | }
36 | | std::vector<int> ans(n + 1);
37 | | for(int i = 1; i <= cc; ++i) {
38 | | | if(dom[i] != sdom[i]) dom[i] = dom[dom[i]];
39 | | | ans[p[i]] = p[dom[i]];
40 | | }
41 | | return ans;
42 | }
43 | };

```

1.10 Tree And Graph

1.10.1 树的计数 Prufer序列

树和其prufer编码一一对应，一颗 n 个点的树，其prufer编码长度为 $n - 2$ ，且度数为 d_i 的点在prufer 编码中出现 $d_i - 1$ 次。

由树得到序列：总共需要 $n - 2$ 步，第 i 步在当前的树中寻找具有最小标号的叶子节点，将与其相连的点的标号设为Prufer序列的第 i 个元素 p_i ，并将此叶子节点从树中删除，直到最后得到一个长度为 $n - 2$ 的Prufer 序列和一个只有两个节点的树。

由序列得到树：先将所有点的度赋初值为 1，然后加上它的编号在Prufer序列中出现的次数，得到每个点的度；执行 $n - 2$ 步，第 i 步选取具有最小标号的度为 1 的点 u 与 $v = p_i$ 相连，得到树中的一条边，并将 u 和 v 的度减一。最后再把剩下的两个度为 1 的点连边，加入到树中。

相关结论： n 个点完全图，每个点度数依次为 $d_1, d_2, \dots, d_n$ ，这样生成树的棵数为： $\frac{(n-2)!}{(d_1-1)!(d_2-1)!\dots(d_n-1)!}$ 。

左边有 n_1 个点，右边有 n_2 个点的完全二分图的生成树棵数为 $n_1^{n_2-1} \times n_2^{n_1-1}$ 。

m 个连通块，每个连通块有 c_i 个点，把他们全部连通的生成树方案数： $(\sum c_i)^{m-2} \prod c_i$

1.10.2 有根树的计数

首先，令 $S_{n,j} = \sum_{1 \leq j \leq n/j}$ ；于是 $n + 1$ 个结点的有根树的总数为 $a_{n+1} = \frac{\sum_{j=1}^n j a_j S_{n-j}}{n}$ 。注： $a_1 = 1, a_2 = 1, a_3 = 2, a_4 = 4, a_5 = 9, a_6 = 20, a_9 = 286, a_{11} = 1842$ 。

1.10.3 无根树的计数

n 是奇数时，有 $a_n - \sum_i^{n/2} a_i a_{n-i}$ 种不同的无根树。

n 是偶数时，有 $a_n - \sum_i^{n/2} a_i a_{n-i} + \frac{1}{2} a_{n/2} (a_{n/2} + 1)$ 种不同的无根树。

1.10.4 生成树计数 Kirchhoff's Matrix-Tree Theorem

Kirchhoff Matrix $T = Deg - A$, Deg 是度数对角阵， A 是邻接矩阵。无向图度数矩阵是每个点度数；有向图度数矩阵是每个点入度。

邻接矩阵 $A[u][v]$ 表示 $u \rightarrow v$ 边个数，重边按照边数计算，自环不计入度数。

无向图生成树计数： $c = |K|$ 的任意1个 $n1$ 阶主子式

有向图外向树计数： $c = |$ 去掉根所在的那阶得到的主子式 $|$

1.10.5 有向图欧拉回路计数 BEST Theorem

$$ec(G) = t_w(G) \prod_{v \in V} (\deg(v) - 1)!$$

其中 $\deg$ 为入度 (欧拉图中等于出度)， $t_w(G)$ 为以 w 为根的外向树的个数。相关计算参考生成树计数。

欧拉连通图中任意两点外向树个数相同： $t_v(G) = t_w(G)$ 。

以 1 结尾的欧拉路径计数就是把 $\deg$ 视为出度，把 $\deg(1)$ 的贡献改为 $\deg(1)!$ 。

1.10.6 Tutte Matrix

Tutte matrix A of a graph $G = (V, E)$:

$$A_{ij} = \begin{cases} x_{ij} & \text{if } (i, j) \in E \text{ and } i < j \\ -x_{ij} & \text{if } (i, j) \in E \text{ and } i > j \\ 0 & \text{otherwise} \end{cases}$$

where x_{ij} are indeterminates. The determinant of this skew-symmetric matrix is then a polynomial (in the variables $x_{ij}, i < j$): this coincides with the square of the pfaffian of the matrix A and is non-zero (as a polynomial) if and only if a perfect matching exists.

1.10.7 Edmonds Matrix

Edmonds matrix A of a balanced ($|U| = |V|$) bipartite graph $G = (U, V, E)$:

$$A_{ij} = \begin{cases} x_{ij} & (u_i, v_j) \in E \\ 0 & (u_i, v_j) \notin E \end{cases}$$

where the x_{ij} are indeterminates. G 有完美匹配当且仅当关于 x_{ij} 的多项式 $\det(A_{ij})$ 不恒为 0. 完美匹配个数等于多项式中单项式的个数.

1.10.8 Erdős-Gallai theorem

度数序列 $d_1 \geq \dots \geq d_n$ 存在对应的简单图当且仅当:

$$\begin{cases} \sum_{i=1}^n d_i \text{ 为偶数} \\ \forall k \in [1, n]: \sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k) \end{cases}$$

2 数论

2.1 取模还原分数

```
1 std::pair<int, int> approx(int p, int q, int A) {
2   | int x = q, y = p, a = 1, b = 0;
3   | for(; x > A;) {
4   |   | std::swap(x, y), std::swap(a, b);
5   |   | a -= x / y * b, x %= y;
6   | }
7   | return {x, a};
8 } //  $q \equiv \frac{x}{a} \pmod{p}, x \leq A, |a|$  取到最小值
```

2.2 扩展欧几里得

结果的 x, y 满足: $ax + by = \gcd(a, b)$; 若 a, b 非 0, 则: $-b \leq x \leq b, -a \leq y \leq a$.

```
1 void exgcd(ll a, ll b, ll & x, ll & y) {
2   | if(!b) return x = 1, y = 0, void();
3   | exgcd(b, a % b, y, x), y -= a / b * x;
4   }
```

2.3 万能欧几里得

```
1 // 万欧
2 // 前提 :  $r < q, r \geq q$  先提几个 U 出来再用
3 // 使用:  $Y * q \leq X * p + r$ , 斜率  $p/q$ , U 表示向上, R 表示到达一个顶点, 先一些 U 再一个 R
4 template<class T>
5 T power(T a, ll k) {
6   | // 有效率需求可以改为半群乘法
7   | if(!k) return T();
8   | T res = a;
9   | for(--k; k;) {
10  |   | if(k & 1) res = res * a;
11  |   | if(k >>= 1) a = a * a;
12  | }
```

```
13 | return res;
14 }
15 template<class T>
16 T solve(ll p, ll q, ll r, ll l, T U, T R) {
17   | if (p >= q)
18   |   | return solve(p % q, q, r, l, U, power(U, p / q) + R);
19   | ll m = ((__int128)p * l + r) / q;
20   | if (!m) return power(R, l);
21   | ll cnt = l - ((__int128)q * m - r - 1) / p;
22   | return power(R, (q - r - 1) / p) + U + solve(q, p, (q - r - 1) % p, m - 1,
23   | R, U) + power(R, cnt);
24 }
```

2.4 floor-sum

$$n < 2^{32}, 1 \leq m < 2^{32}$$

$$result = \sum_{i=0}^{n-1} \lfloor \frac{ai+b}{m} \rfloor \pmod{2^{63}}$$

```
1 u64 floor_sum(u64 n, u64 m, u64 a, u64 b) {
2   | u64 ans = 0;
3   | for(;;) {
4   |   | if(a >= m) ans += n * (n - 1) / 2 * (a / m), a %= m;
5   |   | if(b >= m) ans += n * (b / m), b %= m;
6   |   | u64 ymax = a * n + b; // use u128 if it's big
7   |   | if(ymax < m) break;
8   |   | n = ymax / m;
9   |   | b = ymax % m;
10  |   | std::swap(m, a);
11  | }
12  | return ans;
13 }
```

2.5 Min of Mod of Linear

$$result = \min_{0 \leq i < n} \{(ai + b) \% m\}$$

```
1 int minmod(int n, int m, int a, ll b) {
2   | int ans = m;
3   | for(; n; std::swap(a, m)) {
4   |   | a %= m, b %= m;
5   |   | if(b < 0) b += m;
6   |   | if(b < ans) ans = b;
7   |   | n = (ll)(n - 1) * a + b / m;
8   |   | b -= (ll) m * n;
9   | }
10  | return ans;
11 }
```


2.6 Stern-Brocot Tree 二分

```

1 using cp = std::complex<ll>;
2 cp fracBS(ll n, ll m, auto f) {
3     | bool dir = 1, A = 1, B = 1;
4     | cp lo(0, 1), hi(1, 1); // hi can be (1, 0), f(hi) must be true
5     | if (f(lo)) return lo;
6     | while(A || B) {
7         | ll adv = 0, s = 1;
8         | for (int x = 0; s; (s *= 2) >= x) {
9             | | adv += s;
10            | | cp mid = lo * adv + hi;
11            | | if (mid.real() > n || mid.imag() > m || dir == !f(mid)) {
12                | | | adv -= s, x = 2;
13            | | }
14            | | }
15            | | hi += lo * adv, dir = !dir;
16            | | swap(lo, hi);
17            | | A = B, B = adv;
18        | }
19        | return dir ? hi : lo;
20    } // 返回值是最小的使得 f 为真的
21    // 另外一个是最大的使得 f 为假的

```

2.7 中国剩余定理

返回满足 $x \equiv a_i \pmod{m_i} (i = 1, 2)$ 的最小非负整数 x ，若无解则返回 -1 。

```

1 ll CRT(ll a1, ll p1, ll a2, ll p2) {
2     | ll a, b, gcd = std::gcd(p1, p2);
3     | if ((a1 - a2) % gcd)
4     | | return -1;
5     | exgcd(p1, p2, a, b);
6     | ll k = i128((a2 - a1) % p2 + p2) * (a + p2) % p2;
7     | return p1 / gcd * k + a1;
8 }

```

2.8 Miller-Rabin

调用 checkprime 判断是否为质数。

```

1 using f64 = long double;
2 ll p;
3 f64 invp;
4 void setmod(ll x) {
5     | p = x, invp = (f64) 1 / x;
6 }
7 ll mul(ll a, ll b) {
8     | ll z = a * invp * b + 0.5;
9     | ll res = a * b - z * p;
10    | return res + (res >> 63 & p);
11 }
12 ll pow(ll a, ll x, ll res = 1) {

```

```

13 | for(; x >= 1, a = mul(a, a))
14 | | if(x & 1) res = mul(res, a);
15 | return res;
16 }
17 bool checkprime(ll p) {
18 | if(p == 1) return 0;
19 | setmod(p);
20 | ll d = __builtin_ctzll(p - 1), s = (p - 1) >> d;
21 | for(ll a : {2, 3, 5, 7, 11, 13, 82, 373}) {
22 | | if(a % p == 0)
23 | | | continue;
24 | | ll x = pow(a, s);
25 | | for(int i = 0; i < d; ++i, x = y) {
26 | | | y = mul(x, x);
27 | | | if(y == 1 && x != 1 && x != p - 1)
28 | | | | return 0;
29 | | }
30 | | if(x != 1) return 0;
31 | }
32 | return 1;
33 }

```

2.9 Pollard-rho

调用 factor 获得 x 质因子的可重集合（已从小到大排序）。

```

1 ll rho(ll n) {
2     | if(!(n & 1)) return 2;
3     | static std::mt19937_64 gen(42);
4     | ll x = 0, y = 0, prod = 1, g = gen() % n;
5     | auto f = [&](ll o) { return mul(o, o) + g; };
6     | for(int t = 28, z = 0; t % 32 || std::gcd(prod, n) == 1; ++t) {
7         | | if(ll q = mul(prod, x - y)) prod = q;
8         | | else y = x + ++z, g = gen() % n;
9         | | x = f(x), y = f(f(y));
10    | }
11    | return std::gcd(prod, n);
12 }
13 std::vector<ll> factor(ll x) {
14 | std::vector<ll> res;
15 | auto f = [&](auto f, ll x) {
16 | | if(x == 1) return;
17 | | if(checkprime(x)) return res.push_back(x);
18 | | ll y = rho(x);
19 | | f(f, y), f(f, x / y);
20 | | };
21 | f(f, x), sort(res.begin(), res.end());
22 | return res;
23 }

```

2.10 二次剩余

quadres 用于判断是否存在二次剩余，sqrtp 用于计算结果。 p 或 b 需要是质数（或者 1）。

高斯引理:

$$\left(\frac{a}{p}\right) = (-1)^{\sum_{i=1}^{\frac{p-1}{2}} 1[(ai \bmod p) > p/2]}$$

推论:

$$\begin{cases} \left(\frac{-1}{p}\right) = 1 & \Leftrightarrow p \equiv 1 \pmod{4}, \\ \left(\frac{2}{p}\right) = 1 & \Leftrightarrow p \equiv \pm 1 \pmod{8}, \\ \left(\frac{3}{p}\right) = 1 & \Leftrightarrow p \equiv \pm 1 \pmod{12}, \\ \left(\frac{5}{p}\right) = 1 & \Leftrightarrow p \equiv \pm 1 \pmod{5}. \end{cases}$$

```

1 bool quadres(u64 a, u64 b) {
2   if(a <= 1) return 1;
3   for(;a % 4 == 0;) a /= 4;
4   if(a % 2) return (a % 4 == 1 || b % 4 == 1) == quadres(b % a, a);
5   return (b % 8 == 1 || b % 8 == 7) == quadres(a / 2, b);
6 }
7 int sqrtp(int x, int p) {
8   if(x <= 1) return x;
9   static std::mt19937_64 gen;
10  int w, v, k = (p + 1) / 2;
11  do w = gen() % p; while(quadres(v = ((u64) w * w + p - x) % p, p));
12  using ai = std::array<int, 2>;
13  auto prod = [&](ai x, ai y) {
14    return ai{
15      ((u64) x[0] * y[0] + (u64) x[1] * y[1] % p * v) % p,
16      ((u64) x[0] * y[1] + (u64) x[1] * y[0]) % p
17    };
18  };
19  ai ans{1, 0}, a{w, 1};
20  for(;k >= 1, a = prod(a, a))
21    if(k & 1) ans = prod(ans, a);
22  return ans[0];
23 }

```

3 Math

3.1 拉格朗日反演

$$G(F(x)) = H(x) \Rightarrow [x^n]G(x) = \frac{1}{n}[u^{n-1}]H'(u)\left(\frac{u}{F(u)}\right)^n$$

$$G(F(x)) = x \Rightarrow [x^n]H(G(x)) = \frac{1}{n}[u^{n-1}]H'(u)\left(\frac{u}{F(u)}\right)^n$$

$$G(F(x)) = x \Rightarrow [x^n]G^k(x) = \frac{k}{n}[u^{n-k}]\left(\frac{u}{F(u)}\right)^n$$

3.2 分拆数|五边形数

$$\prod_{i \geq 1} (1 - x^i) = \sum_{k=-\infty}^{\infty} (-1)^k x^{\frac{k(3k-1)}{2}}$$

3.3 Fast Fourier Transform

```

1 using db = double;
2 using cp = std::complex<db>;
3 // cp::real, cp::imag, std::conj, std::arg
4 const db pi = std::acos(-1);
5 int rev[N], lim;
6 cp wn[N];
7 void init(int len) {
8   for(lim = 1; lim < len; lim <= 1);
9   for(static int i = 1; i < lim; i += i) {
10     for(int j = 0; j < i; ++j) {
11       wn[i + j] = std::polar(1., db(j) / i * pi);
12     }
13   }
14   for(int i = 1; i < lim; ++i) {
15     rev[i] = rev[i >> 1] >> 1 | (i % 2u * lim / 2);
16   }
17 }
18 void DFT(cp * a) {
19   for(int i = 0; i < lim; ++i) {
20     if(rev[i] < i) std::swap(a[rev[i]], a[i]);
21   }
22   for(int i = 1; i < lim; i += i) {
23     for(int j = 0; j < lim; j += i + i) {
24       for(int k = 0; k < i; ++k) {
25         cp x = a[i + j + k] * wn[i + k];
26         a[i + j + k] = a[k + j] - x;
27         a[k + j] += x;
28       }
29     }
30   }
31 }
32 void IDFT(cp * a) {
33   DFT(a, std::reverse(a + 1, a + lim));
34   for(int i = 0; i < lim; ++i)
35     a[i] /= lim;
36 }

```

3.4 Number Theoretic Transform

```

1 int rev[N], wn[N], lim, invlim;
2 int pow(int a, int b, int ans = 1) {
3   for(;b >= 1, a = (u64) a * a % mod) if(b & 1)
4     ans = (u64) ans * a % mod;
5   return ans;
6 }
7 void init(int len) {
8   for(lim = 1; lim < len; lim <= 1);
9   invlim = mod - (mod - 1) / lim;
10  for(static int i = 1; i < lim; i += i) {
11    wn[i] = 1;
12    const int w = pow(3, mod / i / 2);
13    for(int j = 1; j < i; ++j) {
14      wn[i + j] = (u64) wn[i + j - 1] * w % mod;
15    }
16  }

```

```

15 | | }
16 | }
17 | for(int i = 1; i < lim; ++i) {
18 | | rev[i] = rev[i >> 1] >> 1 | (i % 2u * lim / 2);
19 | | }
20 | }
21 | void DFT(int * a) {
22 | | static u64 t[N];
23 | | for(int i = 0; i < lim; ++i) t[i] = a[rev[i]];
24 | | for(int i = 1; i < lim; i += i) {
25 | | | if(i >> 19 & 1)
26 | | | | for(int k = 0; k < lim; ++k) t[k] %= mod;
27 | | | for(int j = 0; j < lim; j += i + i) {
28 | | | | for(int k = 0; k < i; ++k) {
29 | | | | | const u64 x = t[i + j + k] * wn[i + k] % mod;
30 | | | | | t[i + j + k] = t[k + j] + (mod - x), t[k + j] += x;
31 | | | | }
32 | | | }
33 | | }
34 | | for(int i = 0; i < lim; ++i) a[i] = t[i] % mod;
35 | }
36 | void IDFT(int * a) {
37 | | DFT(a), std::reverse(a + 1, a + lim);
38 | | for(int i = 0; i < lim; ++i)
39 | | | a[i] = (u64) a[i] * invlim % mod;
40 | }

```

3.5 Generating function

```

1 | void cpy(int * a, int * b, int n) {
2 | | if(a != b) memcpy(a, b, n << 2);
3 | | memset(a + n, 0, (lim - n) << 2);
4 | }
5 | void inv(int * a, int * b, int n) { // b = inv(a) mod x^n
6 | | if(n == 1) return void(*b = pow(*a, mod - 2));
7 | | static int c[N], d[N];
8 | | int m = (n + 1) / 2;
9 | | inv(a, b, m);
10 | | init(n + m), cpy(c, b, m), cpy(d, a, n);
11 | | DFT(c), DFT(d);
12 | | for(int i = 0; i < lim; ++i) c[i] = (u64) c[i] * c[i] % mod * d[i] % mod;
13 | | IDFT(c);
14 | | for(int i = m; i < n; ++i) b[i] = norm(mod - c[i]);
15 | }
16 | void log(int * a, int * b, int n) { // b = log(a) (mod x^n)
17 | | static int c[N], d[N];
18 | | inv(a, c, n), init(n + n);
19 | | for(int i = 1; i < n; ++i) d[i - 1] = (u64) a[i] * i % mod;
20 | | cpy(d, d, n - 1), cpy(c, c, n);
21 | | DFT(c), DFT(d);
22 | | for(int i = 0; i < lim; ++i) c[i] = (u64) c[i] * d[i] % mod;
23 | | IDFT(c), *b = 0;
24 | | for(int i = 1; i < n; ++i) b[i] = pow(i, mod - 2, c[i - 1]);
25 | }

```

3.6 全在线卷积

```

1 | struct oc {
2 | | std::vector<int> f, g, res;
3 | | std::vector<std::vector<int>> fa, fb;
4 | | int n, p;
5 | | oc(int n) : f(n), g(n), res(n), n(n), p(0) {}
6 | | void push(int v0, int v1) {
7 | | | f[p] = v0;
8 | | | res[p] = (res[p] + (u64) f[0] * v1 + (u64) g[0] * v0) % mod;
9 | | | g[p++] = v1;
10 | | | static int A[N], B[N];
11 | | | int lb = p & -p;
12 | | | init(lb * 2);
13 | | | memset(A, 0, lim << 2);
14 | | | memset(B, 0, lim << 2);
15 | | | for(int i = 0; i < lb; ++i) A[i] = g[p - lb + i], B[i] = f[p - lb + i];
16 | | | DFT(A), DFT(B);
17 | | | if(lb == p) {
18 | | | | fa.emplace_back(A, A + lim);
19 | | | | fb.emplace_back(B, B + lim);
20 | | | | for(int i = 0; i < lim; ++i) A[i] = (u64) A[i] * B[i] % mod;
21 | | | | } else {
22 | | | | auto & C = fb[std::lg(lim)], & D = fa[std::lg(lim)];
23 | | | | for(int i = 0; i < lim; ++i) A[i] = ((u64) A[i] * C[i * 2] + (u64)
24 | | | | | B[i] * D[i * 2]) % mod;
25 | | | }
26 | | | IDFT(A);
27 | | | for(int j = p; j < p + lb && j < n; ++j) res[j] = (res[j] + A[j - p +
28 | | | lb]) % mod;
29 | | }
30 | | };
31 | struct Exp : oc {
32 | | std::vector<int> res;
33 | | Exp(int n) : oc(n), res(n) {}
34 | | void push(int v) {
35 | | | if(!res[0]) return void(res[0] = 1);
36 | | | oc::push(res[p], v * u64(p + 1) % mod);
37 | | | res[p] = (u64) oc::res[p - 1] * inv[p] % mod;
38 | | }
39 | | };
40 | struct Ln : oc {
41 | | std::vector<int> res; int fi;
42 | | Ln(int n) : oc(n), res(n), fi(0) {}
43 | | void push(int v) {
44 | | | if(!fi) return void(fi = 1);
45 | | | oc::push(res[p] * (u64) p % mod, v);
46 | | | res[p] = ((u64) v * p + mod - oc::res[p - 1]) % mod * inv[p] % mod;
47 | | }
48 | | };
49 | struct Inv : oc {
50 | | std::vector<int> res; int fi;
51 | | Inv(int n) : oc(n), res(n), fi(0) {}
52 | | void push(int v) {

```

```

51 |   | res[p] = fi ? (oc::res[p] + (u64) v * res[0]) % mod * (mod - res[0]) %
    |   | mod : pow(fi = v, mod - 2);
52 |   | oc::push(res[p], v);
53 |   }
54 | };

```

3.7 多项式复合

$res(x) = f(g(x)) \bmod x^n$, $g(0) = 0, \deg(f), \deg(g) < n$.

```

1  using fps = std::vector<int>;
2  fps comp(fps f, fps g, int n) {
3      | int S = 4 << std::__lg((n - 1) | 1);
4      | f.resize(S), g.resize(S * 2);
5      | for(int i = S / 2 - 1; i >= 0; --i) {
6      |   | f[i * 2] = f[i], f[i * 2 + 1] = 0;
7      |   | g[i] = g[i] ? mod - g[i] : 0;
8      | }
9      | auto dc = [&](auto dc, fps & P, fps & Q, int d, int n) {
10     |   | if(n == 1) return;
11     |   | fps V(S * 2);
12     |   | for(int i = 0; i < S; ++i) if(i & n) Q[i] = 0;
13     |   | Q[S] = 1, init(S * 2), DFT(data(Q));
14     |   | for(int i = 0; i < S; ++i) V[i] = (u64) Q[i] * Q[i + S] % mod;
15     |   | init(S), IDFT(data(V)), V[0] = 0;
16     |   | dc(dc, P, V, d * 2, n / 2);
17     |   | init(S), DFT(data(P));
18     |   | for(int k = 0; k < S; ++k) {
19     |   |   | int u = Q[k], v = Q[k + S], w = P[k];
20     |   |   | Q[k] = (u64) v * w % mod;
21     |   |   | Q[k + S] = (u64) u * w % mod;
22     |   | }
23     |   | init(S * 2), IDFT(data(Q));
24     |   | for(int i = 0; i < S; ++i) P[i] = (i & n) ? 0 : Q[i + S];
25     | };
26     | dc(dc, f, g, 1, S / 2);
27     | f.resize(n);
28     | return f;
29 }

```

3.8 Power Projection

$\deg(f) = \deg(g) = n - 1, g(0) = 0$, 返回 $ans[i] = [x^{n-1}]f(x)g(x)^i, i = 0, \dots, n - 1$

```

1  fps PowerProjection(fps f, fps g, int n) {
2      | int m = 2 << std::__lg((n - 1) | 1), S = m * 2;
3      | reverse(f.begin(), f.end());
4      | f.insert(f.begin(), m + S, 0);
5      | f.resize(S * 2);
6      | g.resize(S * 2);
7      | for(int i = 0; i < m; ++i) g[i] = g[i] ? mod - g[i] : 0;
8      | for(; m > 1; m /= 2) {

```

```

9      |   | for(int i = 0; i < S * 2; ++i) (i & m ? g[i] : f[i]) = 0;
10     |   | for(int i = 0; i < S; ++i) g[i + S] = 0, f[i] = 0;
11     |   | g[S] = 1, init(S * 2), IDFT(f.data()), DFT(g.data());
12     |   | for(int i = 0; i < S; ++i) {
13     |   |   | u64 a = f[i], b = f[i + S];
14     |   |   | u64 c = g[i], d = g[i + S];
15     |   |   | f[i + S] = (a * d + b * c) % mod;
16     |   |   | g[i] = (u64)c * d % mod;
17     |   | }
18     |   | init(S), DFT(f.data() + S), IDFT(g.data()), g[0] = 0;
19     | };
20     | for(int i = 0; i < n; ++i) f[i] = f[i * 2 + 1 + S];
21     | f.resize(n);
22     | return f;
23 }

```

3.9 常数线性递推

3.9.1 Berlekamp Massey

返回最短的 c 使得: $\forall n \geq \text{size}(c) : a_n = \sum_{i=0}^{\text{size}(c)-1} c_i a_{n-1-i}$

```

1  std::vector<int> BM(std::vector<int> a) {
2      | int n = size(a), len = 0, s = 1, m = 0;
3      | std::vector<int> res(n), lst(n), tmp;
4      | res[0] = 1;
5      | for(int i = 0; i < n; ++i) {
6      |   | int d = 0; ++m;
7      |   | for(int j = 0; j <= len; ++j)
8      |   |   | d = (d + (u64) res[j] * a[i - j]) % mod;
9      |   | if(!d) continue;
10     |   | tmp = res;
11     |   | u64 k = pow(s, mod - 2, mod - d);
12     |   | for(int j = m; j < n; ++j) res[j] = (res[j] + k * lst[j - m]) % mod;
13     |   | if(len * 2 > i) continue;
14     |   | len = i - len + 1, lst = tmp, s = d, m = 0;
15     | }
16     | std::vector<int> c(len);
17     | for(int i = 0; i < len; ++i) c[i] = (mod - res[i + 1]) % mod;
18     | return c;
19 }

```

3.9.2 递推式转分式

$$a_i = \sum_{j=1}^d c_j a_{i-j}, i \geq d \Leftrightarrow a_i = [x^i] \frac{p(x)}{q(x)}$$

```

1  std::pair<fps, fps> convert(fps a, fps c) {
2      | int d = c.size();
3      | fps q(d + 1); q[0] = 1;
4      | for(int i = 0; i < d; ++i) q[i + 1] = norm(mod - c[i]);

```

```

5 | for(;q.back() == 0;) q.pop_back();
6 | fps p = conv(a, q);
7 | p.resize(a.size());
8 | return {p, q};
9 | }

```

3.9.3 Bostan-Mori

$$[x^k] \frac{p(x)}{q(x)}$$

```

1 | int bostan_mori(fps p, fps q, ll k) {
2 |     int n = q.size();
3 |     for(;k >= 1) {
4 |         auto nq = q;
5 |         for(int i = 1; i < n; i += 2) nq[i] = mod - nq[i];
6 |         auto u = conv(p, nq), v = conv(q, nq);
7 |         int t = k & 1, m = (size(u) + 1 - t) / 2;
8 |         p.resize(m);
9 |         for(int i = 0; i < m; ++i) p[i] = u[i * 2 + t];
10 |        for(int i = 0; i < n; ++i) q[i] = v[i * 2];
11 |    }
12 |    return pow(q[0], mod - 2, p[0]);
13 | }

```

3.9.4 远处区间值

$$[x^{[k,k+m]}] \frac{p(x)}{q(x)}$$

如果 $\deg(p) \geq \deg(q)$, 请使用 solve_x, 不然可以直接使用 solve。

```

1 | fps inv(fps a, int n) {
2 |     static int res[N];
3 |     if(a.size() < n) a.resize(n);
4 |     inv(a.data(), res, n);
5 |     return fps(res, res + n);
6 | }
7 | fps cut(const fps & f, int d) {
8 |     return fps(f.begin(), f.begin() + std::min((int)f.size(), d));
9 | }
10 | fps revp(const fps & f) { return fps(f.rbegin(), f.rend()); }
11 | std::pair<fps, fps> polydiv(fps f, fps g) {
12 |     int n = size(f), gd = size(g) - 1, qd = n - gd;
13 |     if(n <= gd) return {fps(), f};
14 |     auto F = cut(revp(f), qd);
15 |     auto G = inv(revp(g), qd);
16 |     fps q = revp(cut(conv(F, G), qd));
17 |     g = conv(g, q);
18 |     for(int i = 0; i < gd; ++i) f[i] = (f[i] + mod - g[i]) % mod;
19 |     return {q, cut(f, gd)};
20 | }

```

```

21 | fps solve(fps p, fps q, u64 k, int m) {
22 |     const int d = size(q) - 1;
23 |     auto rec = [&](auto rec, ll k) {
24 |         if(k == 0) return d > 0 ? fps{1} : fps();
25 |         if(k == 1) {
26 |             int p = pow(q[0], mod - 2); fps res(d);
27 |             for(int i = 0; i < d; ++i) res[i] = u64(mod - p) * q[i + 1] % mod;
28 |             return res;
29 |         }
30 |         fps f = rec(rec, (k + 1) / 2);
31 |         f = conv(f, f);
32 |         if(k & 1) f.insert(f.begin(), 0);
33 |         return polydiv(f, q).second;
34 |     };
35 |     fps res = rec(rec, k);
36 |     res = polydiv(conv(res, p), q).second;
37 |     res = conv(res, inv(q, m));
38 |     return fps(res.begin(), res.begin() + m);
39 | }
40 | fps solve_x(fps p, fps q, u64 k, int m) {
41 |     auto [a, p0] = polydiv(p, q);
42 |     auto ans = p0.size() ? solve(p0, q, k, m) : fps(m);
43 |     for(int i = 0; i < m; ++i) {
44 |         if(k + i < (int) a.size()) ans[i] = (ans[i] + a[i + k]) % mod;
45 |     }
46 |     return ans;
47 | }

```

3.10 线性规划 (单纯形法)

$x[1 \cdots n]$ 是变量, $a[1 \cdots m][\cdots]$ 是约束。

所有约束为:

$$\forall i \in [1, m] : a[i][0] + \sum_{j=1}^n a[i][j]x[j] \geq 0$$

$$\forall j \in [1, n] : x[j] \geq 0$$

最大化

$$\sum_{j=1}^n a[0][j]x[j]$$

如果无解返回 nan, 如果无界返回 inf. 最后的 x 里存放的是最优解。

```

1 | using db = long double;
2 | const db eps = 1e-16;
3 | int sgn(db x) { return x < -eps ? -1 : x > eps; }
4 | namespace LP {
5 |     const int N = 21, M = 21;
6 |     int n, m;
7 |     db a[M + N][N], x[N + M];
8 |     int id[N + M];
9 |     void pivot(int p, int o) {

```

```

10 | | std::swap(id[p], id[o + n]);
11 | | db w = -a[o][p];
12 | | for(int i = 0; i <= n; ++i) a[o][i] /= w;
13 | | a[o][p] = -1 / w;
14 | | for(int i = 0; i <= m; ++i) if(sgn(a[i][p]) && i != o) {
15 | | | db w = a[i][p]; a[i][p] = 0;
16 | | | for(int j = 0; j <= n; ++j) a[i][j] += w * a[o][j];
17 | | }
18 | }
19 | db solve() { // nan : 无解, inf : 无界, 否则返回最大值
20 | | for(int i = 1; i <= n + m; ++i) id[i] = i;
21 | | for(;;) {
22 | | | int p = 0, min = 1;
23 | | | for(int i = 1; i <= m; ++i) {
24 | | | | if(a[i][0] < a[min][0]) min = i;
25 | | | }
26 | | | if(a[min][0] >= -eps) break;
27 | | | for(int i = 1; i <= n; ++i) if(a[min][i] > eps && id[i] > id[p]) {
28 | | | | p = i;
29 | | | }
30 | | | if(!p) return nan("");
31 | | | pivot(p, min);
32 | | }
33 | | for(;;) {
34 | | | int p = 1;
35 | | | for(int i = 1; i <= n; ++i) if(a[0][i] > a[0][p]) p = i;
36 | | | if(a[0][p] < eps) break;
37 | | | db min = INFINITY; int o = 0;
38 | | | for(int i = 1; i <= m; ++i) if(a[i][p] < -eps) {
39 | | | | db w = -a[i][0] / a[i][p]; int d = sgn(w - min);
40 | | | | if(d < 0 || !d && id[i] > id[o]) o = i, min = w;
41 | | | }
42 | | | if(!o) return INFINITY;
43 | | | pivot(p, o);
44 | | }
45 | | for(int i = 1; i <= m; ++i) x[id[i] + n] = a[i][0];
46 | | return a[0][0];
47 | }
48 | }

```

3.11 黄金三分

返回 f 在 $[min(a, c), max(a, c)]$ 上的极大值点 (f 需要是单峰函数)。

```

1 | db findmax(db a, db c, auto f) {
2 | | auto g = [8](db l, db r) {
3 | | | return l + (r - l) * (std::numbers::phi_v<db> - 1);
4 | | };
5 | | db b = g(a, c), bv = f(b);
6 | | for(int i = 0; i < 45; ++i) {
7 | | | db x = g(a, b), xv = f(x);
8 | | | if(xv > bv) { // change here if findmin
9 | | | | c = b, b = x, bv = xv;
10 | | | } else {

```

```

11 | | | a = c, c = x;
12 | | }
13 | | }
14 | | return bv;
15 | } // log1.618(2) ≈ 1.44

```

3.12 特征多项式

$ans_i = [\lambda^i] \det(\lambda I - A)$ 。

```

1 | std::vector<int> charpoly(mat a) {
2 | | int n = a.size();
3 | | for(int i = 1; i < n; ++i) {
4 | | | for(int j = i; j < n; ++j) {
5 | | | | if(a[j][i - 1]) {
6 | | | | | std::swap(a[i], a[j]);
7 | | | | | for(int k = 0; k < n; ++k) std::swap(a[k][i], a[k][j]);
8 | | | | | break;
9 | | | | }
10 | | | }
11 | | | if(!a[i][i - 1]) continue;
12 | | | int x = a[i][i - 1], inv = pow(x, mod - 2);
13 | | | for(int j = 0; j < n; ++j) a[j][i] = a[j][i] * (u64)x % mod;
14 | | | for(int j = i - 1; j < n; ++j) a[i][j] = a[i][j] * (u64)inv % mod;
15 | | | for(int j = i + 1; j < n; ++j) {
16 | | | | int z = a[j][i - 1], iz = mod - z;
17 | | | | for(int k = 0; k < n; ++k) a[k][i] = (a[k][i] + (u64) a[k][j] * z) %
18 | | | | mod;
19 | | | | for(int k = i - 1; k < n; ++k) a[j][k] = (a[j][k] + (u64) a[i][k] *
20 | | | | iz) % mod;
21 | | | }
22 | | }
23 | | mat dp(n + 1, std::vector<int>(n + 1));
24 | | dp[0][0] = 1;
25 | | for(int i = 0; i < n; ++i) {
26 | | | int x = 1;
27 | | | for(int j = i; j >= 0; --j) {
28 | | | | int y = u64(mod - x) * a[j][i] % mod;
29 | | | | for(int k = 0; k <= j; ++k) {
30 | | | | | dp[i + 1][k] = ((u64) y * dp[j][k] + dp[i + 1][k]) % mod;
31 | | | | }
32 | | | | if(j) x = (u64) x * a[j][j - 1] % mod;
33 | | | }
34 | | | for(int k = 0; k <= i; ++k) {
35 | | | | dp[i + 1][k + 1] = (dp[i][k] + dp[i + 1][k + 1]) % mod;
36 | | | }
37 | | }
38 | | return dp[n];
39 | }

```

3.13 $\det(A + Bx)$

$ans_i = [x^i] \det(A + Bx)$, 要抄特征多项式。

```

1 std::vector<int> det(mat a, mat b) {
2     int n = a.size();
3     std::vector<int> ans(n+1);
4     int t = 1, d = 0;
5     for(int i = 0; i < n; ++i) {
6         for(; !b[i][i];) {
7             for(int j = i + 1; j < n; ++j) if(b[i][j]) {
8                 t = mod - t;
9                 for(int k = 0; k < n; ++k) std::swap(a[k][i], a[k][j]),
10                    std::swap(b[k][i], b[k][j]);
11                 break;
12             }
13             if(b[i][i]) break;
14             if(++d > n) return ans;
15             for(int j = 0; j < n; ++j) {
16                 b[i][j] = a[i][j], a[i][j] = 0;
17             }
18             for(int j = 0; j < i; ++j) {
19                 u64 c = mod - b[i][j];
20                 for(int k = 0; k < n; ++k) {
21                     a[i][k] = (a[i][k] + c * a[j][k]) % mod;
22                     b[i][k] = (b[i][k] + c * b[j][k]) % mod;
23                 }
24             }
25             t = (u64) t * b[i][i] % mod;
26             u64 inv = pow(b[i][i], mod - 2);
27             for(int j = 0; j < n; ++j) {
28                 a[i][j] = a[i][j] * inv % mod;
29                 b[i][j] = b[i][j] * inv % mod;
30             }
31             for(int j = 0; j < n; ++j) if(j != i) {
32                 u64 c = mod - b[j][i];
33                 for(int k = 0; k < n; ++k) {
34                     a[j][k] = (a[j][k] + c * a[i][k]) % mod;
35                     b[j][k] = (b[j][k] + c * b[i][k]) % mod;
36                 }
37             }
38         }
39         auto r = charpoly(a);
40         for(int i = d; i <= n; ++i) ans[i - d] = u64((n - i) % 2 ? mod - t : t) *
41             r[i] % mod;
42         return ans;
43     }
44 }

```

4 字符串

4.1 后缀自动机 SAM

需要两倍点数量。

```

1 int c[N][26], mx[N], fail[N], tot = 1;
2 int append(int id, int w) {

```

```

3     int p = id, now = ++ tot;
4     //right[now] = id;
5     for(mx[now] = mx[p] + 1; p && !c[p][w]; p = fail[p])
6         c[p][w] = now;
7     if(!p) fail[now] = 1;
8     else {
9         int q = c[p][w];
10        if(mx[q] == mx[p] + 1) fail[now] = q;
11        else {
12            int x = ++ tot; mx[x] = mx[p] + 1;
13            memcpy(c[x], c[q], sizeof(c[0])), fail[x] = fail[q]; //right[x] =
14                right[q];
15            for(fail[q] = fail[now] = x; p && c[p][w] == q; p = fail[p])
16                c[p][w] = x;
17        }
18    }
19    return now;
20 }
21 void buildtree() { // 倒着建
22     for(int i = 2; i <= tot; ++i)
23         son[fa[i]][s[right[i] + mx[fa[i]]] - 'a'] = i;
24 }

```

4.2 回文自动机 PAM

```

1 int c[N][26], fail[N], len[N], tot;
2 void init() {
3     fail[0] = 1, len[++tot] = -1;
4     // root is 1
5 }
6 int get_fail(int o, char * x) {
7     for(; *x != x[-len[o] - 1];)
8         o = fail[o];
9     return o;
10 }
11 int append(int o, char * x) {
12     o = get_fail(o, x);
13     int & p = c[o][*x - 'a'];
14     if(!p) {
15         fail[++tot] = c[get_fail(fail[o], x)][*x - 'a'];
16         len[p = tot] = len[o] + 2;
17     }
18     return p;
19 }

```

4.3 回文自动机 PAM dp

atom 要赋值，然后 query 是 pam id, seq id, 都要问。结果是 $\sum_{s[i:seqid+1] \text{ is palindrome}} atom[i-1]$

```

1 info atom[N];
2 int c[N][26], fail[N], len[N], tot;
3 int diff[N], bigfail[N];
4 void init() {

```



```

5   bigfail[0] = fail[0] = 1, len[++tot] = -1;
6   // root is 1
7 }
8 int get_fail(int o, char * x) {
9     for(; *x != x[-len[o] - 1];) {
10         o = (*x != x[-len[fail[o]] - 1] ? bigfail : fail)[o];
11     }
12     return o;
13 }
14 int append(int o, char * x) {
15     o = get_fail(o, x);
16     int & p = c[o][*x - 'a'];
17     if(!p) {
18         fail[++tot] = c[get_fail(fail[o], x)][*x - 'a'];
19         len[p = tot] = len[o] + 2;
20         diff[p] = len[p] - len[fail[p]];
21         bigfail[p] = diff[p] == diff[fail[p]] ? bigfail[fail[p]] : fail[p];
22     }
23     return p;
24 }
25 info node[N];
26 info query(int x, int i) {
27     info z = {};
28     for(; x = bigfail[x]) {
29         const int go = bigfail[x], fa = fail[x];
30         info & t = node[x] = atom[i - len[go] - diff[x]];
31         if(fa != go) t = node[fa] + t;
32         z = t + z;
33     }
34     return z;
35 }
36

```

4.4 AC 自动机

```

1  const int sig = 26;
2  int son[N][sig], fail[N], cnt;
3  int ins(const char * c) {
4      int x = 0;
5      for(; *c; ++c) {
6          int & p = son[x][*c - 'a'];
7          if(!p) p = ++cnt;
8          x = p;
9      }
10     return x;
11 }
12 void build_ac() {
13     std::queue<int> Q;
14     for(int i = 0; i < sig; ++i) if(son[0][i]) Q.push(son[0][i]);
15     for(; Q.size(); ) {
16         int x = Q.front(); Q.pop();
17         for(int i = 0; i < sig; ++i)
18             if(son[x][i]) fail[son[x][i]] = son[fail[x]][i], Q.push(son[x][i]);
19             else son[x][i] = son[fail[x]][i];
20     }
21 }

```

```

21 }

```

4.5 SA

后缀数组, sa 是个 $0 \sim n-1$ 的排列, $h[i] = lcp(s[sa[i]:], s[sa[i+1]:])$ 。

```

1  auto SA(const std::vector<int> & s) {
2      int n = s.size();
3      std::vector<int> sa(n), gap(n), rk(n), h(n-1);
4      std::vector<std::array<int, 2>> o(n);
5      gap[0] = 1;
6      for(int i = 0; i < n; ++i) o[i] = {s[i], i};
7      for(int L = 1; L <= 2) {
8          for(int p = n, i = n-1; i >= 0; --i) if(gap[i]) {
9              sort(o.begin() + i, o.begin() + p, [](auto u, auto v) { return u[0] < v[0]; });
10             for(; --p > i;) gap[p] = o[p][0] != o[p-1][0];
11         }
12         int s = 0;
13         for(int i = 0; i < n; ++i) rk[o[i][1]] = s += gap[i];
14         if(s == n) break;
15         for(auto & [v, p] : o) v = p + L < n ? rk[p + L] : 0;
16     }
17     for(int i = 0; i < n; ++i) sa[i] = o[i][1];
18     for(int i = 0, k = 0; i < n; ++i) if(rk[i] != n) {
19         int j = sa[rk[i]];
20         for(k -= !!k; i + k < n && j + k < n && s[i + k] == s[j + k]; ++k);
21         h[rk[i] - 1] = k;
22     }
23     return std::pair(sa, h);
24 }

```

4.6 Z algorithm

返回的 $lcp[i] = lcp(s, s[i:])$ 。

```

1  std::vector<int> Zalgo(const std::string & s) {
2      std::vector<int> lcp(s.size());
3      lcp[0] = s.size();
4      for(int i = 1, l = 0, r = 0; i < (int) s.size(); ++i) {
5          int & x = lcp[i];
6          if(i < r) x = std::min(lcp[i-l], r-i);
7          for(; i+x < (int) s.size() && s[i] == s[i+x];)
8              ++x;
9          if(i+x > r) l = i, r = i+x;
10     }
11     return lcp;
12 }

```

4.7 Manacher

返回每个回文中心的回文半径大小, 奇偶分开考虑。

```

1 std::vector<int> manacher(const std::string & s, bool is_even) {
2     const int N = s.size() - is_even;
3     std::vector<int> o(N); // 半径
4     for(int i = 0, mid = 0, r = 0; i < N; ++i) {
5         int & x = o[i] = 0;
6         if(i < r) x = std::min(r - i, o[mid - i]);
7         for(; i - x >= 0 && i + x < N && s[i - x] == s[i + x + is_even];)
8             ++ x;
9         if(i + x > r) {
10             mid = i * 2;
11             r = i + x;
12         }
13     }
14     return o;
15 }

```

4.8 KMP

$s.size() > 0$, b_i 为最大的 $j < i$, 使得 $s[:j] = s[i-j:i]$

```

1 std::vector<int> kmp(const std::string & s) {
2     std::vector<int> b(s.size());
3     for(int i = 1, j = 0; i < (int) s.size(); ++i) {
4         for(; j && s[i] != s[j];) j = b[j - 1];
5         b[i] = j += s[i] == s[j];
6     }
7     return b;
8 }
9 std::vector<int> match(const std::string & s, const std::string & t) {
10     auto b = kmp(s);
11     std::vector<int> ans;
12     for(int i = 0, j = 0; i < (int) t.size(); ++i) {
13         for(; j && t[i] != s[j];) j = b[j - 1];
14         j += t[i] == s[j];
15         if(j == (int) s.size()) ans.push_back(i - s.size() + 1), j = b[j - 1];
16     }
17     return ans;
18 }

```

4.9 最小表示法

返回 s 的最小表示的起始位置。可以用 `rotate` 算出最小表示的字符串。

```

1 int minrep(const std::vector<int> & s) {
2     int k = 0, i = 0, j = 1, n = size(s);
3     for(; k < n && i < n && j < n;) {
4         if(int d = s[(i + k) % n] - s[(j + k) % n]) {
5             (d > 0 ? i : j) += k + 1;
6             i += i == j, k = 0;
7         } else {
8             ++ k;
9         }
10    }

```

```

10    }
11    return std::min(i, j);
12 } // rotate(s.begin(), s.begin() + minrep(s), s.end());

```

4.10 Lyndon 分解

```

1 auto duval(const std::vector<int> & s) {
2     int n = s.size();
3     std::vector<std::vector<int>> res;
4     for(int i = 0; i < n;) {
5         int j = i + 1, k = i;
6         for(; j < n && s[k] <= s[j]; ++j) {
7             if(s[k] < s[j]) {
8                 k = i;
9             } else {
10                ++ k;
11            }
12        }
13        for(; i <= k; i += j - k) {
14            res.emplace_back(s.begin() + i, s.begin() + i + (j - k));
15        }
16    }
17    return res;
18 }

```

5 数据结构

5.1 区间加区间求和树状数组

add 区间加, query 区间求和。区间都是闭区间。

```

1 // 后缀加, 前缀求和
2 struct BIT {
3     ll a[N], b[N];
4     void add(ll p, int v) {
5         for(int i = p; i < N; i += i & -i)
6             a[i] += v, b[i] += p * v;
7     }
8     ll qry(ll p) {
9         ll res = 0;
10        for(int i = p; i &= i - 1) res += (p + 1) * a[i] - b[i];
11        return res;
12    }
13    void add(int l, int r, int v) { add(l, v), add(r + 1, -v); }
14    ll qry(int l, int r) { return qry(r) - qry(l - 1); }
15 } bit;

```

5.2 zkw 线段树

单点修改区间半群。如果半群没有单位元可以用 `qry2`。

```

1 struct seg {

```

```

2 | ll o[1 << 20]; int L;
3 | void upt(int x) {
4 | | o[x] = o[x << 1] + o[x << 1 | 1];
5 | }
6 | void init(int n, int * w) {
7 | | L = 2 << std::__lg(n + 1);
8 | | for(int i = 1; i <= n; ++i) o[i + L] = w[i];
9 | | for(int i = L; i >= 1; --i) upt(i);
10 | }
11 | void upt(int p, int v) {
12 | | for(o[p += L] += v; p >>= 1; upt(p));
13 | }
14 | ll qry(int l, int r) {
15 | | l += L - 1, r += L + 1;
16 | | ll ans = 0;
17 | | for(; l ^ r ^ 1; l >>= 1, r >>= 1) {
18 | | | if((l & 1) == 0) ans += o[l ^ 1];
19 | | | if((r & 1) == 1) ans += o[r ^ 1];
20 | | }
21 | | return ans;
22 | }
23 | ll qry2(int l, int r) {
24 | | if(l == r) return o[l + L];
25 | | ll le = o[l + L], ri = o[r + L];
26 | | l += L, r += L;
27 | | for(; l ^ r ^ 1; l >>= 1, r >>= 1) {
28 | | | if((l & 1) == 0) le = le + o[l ^ 1];
29 | | | if((r & 1) == 1) ri = o[r ^ 1] + ri;
30 | | }
31 | | return le + ri;
32 | }
33 | } sgt;

```

5.3 静态区间半群

$O(n \log n)$ 预处理, $O(1)$ 查询。

```

1 | template<class T> struct rq {
2 | | std::vector<std::vector<info>> v; // change type here
3 | | T op;
4 | | rq(auto & a, T op) : v(1, a), op(op) {
5 | | | int n = a.size();
6 | | | for(int d = 2; d < n; d += d) {
7 | | | | auto & o = v.emplace_back(a);
8 | | | | for(int j = d; j < n; j += d + d) {
9 | | | | | for(int k = 1; k < d; ++k) {
10 | | | | | | if(j + k < n) o[j + k] = op(o[j + k - 1], o[j + k]);
11 | | | | | | o[j - k - 1] = op(o[j - k - 1], o[j - k]);
12 | | | | | }
13 | | | | }
14 | | | }
15 | | }
16 | | auto query(int l, int r) {
17 | | | if(l == r) return v[0][l];

```

```

18 | | | const int lg = std::__lg(l ^ r);
19 | | | return op(v[lg][l], v[lg][r]);
20 | | }
21 | };

```

5.4 Link Cut Tree

```

1 | int son[N][2], fa[N], rev[N];
2 | int get(int x, int p = 1) { return son[fa[x]][p] == x; }
3 | void update(int x) { }
4 | int is_root(int x) { return !(get(x) || get(x, 0)); }
5 | void rotate(int x) {
6 | | int y = fa[x], z = fa[y], b = get(x);
7 | | if(!is_root(y)) son[z][get(y)] = x;
8 | | son[y][b] = son[x][!b], son[x][!b] = y;
9 | | fa[son[y][b]] = y, fa[y] = x, fa[x] = z;
10 | | update(y);
11 | }
12 | void put(int x) {
13 | | if(x) rev[x] ^= 1, std::swap(son[x][0], son[x][1]);
14 | }
15 | void down(int x) {
16 | | if(rev[x]) {
17 | | | put(son[x][0]);
18 | | | put(son[x][1]);
19 | | | rev[x] = 0;
20 | | }
21 | }
22 | void pushdown(int x) {
23 | | if(!is_root(x)) pushdown(fa[x]);
24 | | down(x);
25 | }
26 | void splay(int x) {
27 | | for(pushdown(x); !is_root(x); rotate(x)) if(!is_root(fa[x]))
28 | | | rotate(get(x) ^ get(fa[x]) ? x : fa[x]);
29 | | update(x);
30 | }
31 | void access(int x) {
32 | | for(int t = 0; x; son[x][1] = t, t = x, x = fa[x])
33 | | | splay(x);
34 | }
35 | void makeroot(int x) {
36 | | access(x), splay(x), put(x);
37 | }

```

5.5 压位 Trie

```

1 | int ctz(u64 x) { return __builtin_ctzll(x); }
2 | int clz(u64 x) { return __builtin_clzll(x); }
3 | struct ds {
4 | | std::vector<std::vector<u64>> o;
5 | | int d;
6 | | ds(int s) { // [0, s]
7 | | | do o.emplace_back((s >>= 6) + 1); while(s);

```

```

8 | | d = o.size();
9 | }
10 | void ins(int x) {
11 | | for(int i = 0; i < d; ++i) {
12 | | | u64 w = 1ull << (x & 63), & v = o[i][x >= 6];
13 | | | if(v & w) return ;
14 | | | v |= w;
15 | | }
16 | }
17 | void del(int x) {
18 | | for(int i = 0; i < d; ++i) {
19 | | | u64 w = 1ull << (x & 63), & v = o[i][x >= 6];
20 | | | if(v &= ~w) return ;
21 | | }
22 | }
23 | int next(int x) const {
24 | | for(int i = 0; i < d; ++i) {
25 | | | int b = x & 63;
26 | | | if(u64 v = o[i][x >= 6] >> b >> 1) {
27 | | | | int ans = x * 64 + ctz(v) + b + 1;
28 | | | | for(int j = i - 1; j >= 0; --j)
29 | | | | | ans = ans * 64 + ctz(o[j][ans]);
30 | | | | return ans;
31 | | | }
32 | | }
33 | | return -1;
34 | }
35 | int prev(int x) const {
36 | | for(int i = 0; i < d; ++i) {
37 | | | int b = x & 63;
38 | | | if(u64 v = o[i][x >= 6] & ((1ull << b) - 1)) {
39 | | | | int ans = x * 64 + (63 - clz(v));
40 | | | | for(int j = i - 1; j >= 0; --j) {
41 | | | | | ans = ans * 64 + (63 - clz(o[j][ans]));
42 | | | | }
43 | | | | return ans;
44 | | | }
45 | | }
46 | | return -1;
47 | }
48 | int min() const {
49 | | int ans = 0;
50 | | for(int j = d - 1; j >= 0; --j) ans = ans * 64 + ctz(o[j][ans]);
51 | | return ans;
52 | }
53 | int max() const {
54 | | int ans = 0;
55 | | for(int j = d - 1; j >= 0; --j) ans = ans * 64 + (63 - clz(o[j][ans]));
56 | | return ans;
57 | }
58 | };

```

5.6 pbds tree

```

1 | #include <bits/extc++.h>

```

```

2 | using namespace __gnu_pbds;
3 | template<class T> // insert, erase, join, order_of_key, find_by_order(return
  iterator), order is 0-index
4 | using Tree = tree<T, null_type, std::less<T>, rb_tree_tag,
  tree_order_statistics_node_update>;

```

6 geometry

6.1 向量

```

1 | using db = long double;
2 | const db eps = 1e-10;
3 | db sgn(db x) { return x < -eps ? -1 : x > eps; }
4 | db eq(db x, db y) { return !sgn(x - y); }
5 | struct p2 {
6 | | db x, y;
7 | | db norm() const { return x * x + y * y; }
8 | | db abs() const { return std::sqrt(x * x + y * y); }
9 | | db arg() const { return atan2(y, x); }
10 | };
11 | db arg(p2 x, p2 y) {
12 | | db a = y.arg() - x.arg();
13 | | if(a > pi) a -= pi * 2;
14 | | if(a < -pi) a += pi * 2;
15 | | return a;
16 | }
17 | p2 r90(p2 x) { return {-x.y, x.x}; }
18 | p2 operator + (p2 x, p2 y) { return {x.x + y.x, x.y + y.y}; }
19 | p2 operator - (p2 x, p2 y) { return {x.x - y.x, x.y - y.y}; }
20 | p2 operator / (p2 x, db y) { return {x.x / y, x.y / y}; }
21 | p2 operator * (p2 x, db y) { return {x.x * y, x.y * y}; }
22 | p2 operator * (db y, p2 x) { return {x.x * y, x.y * y}; }
23 | db operator * (p2 x, p2 y) { return x.x * y.y - x.y * y.x; }
24 | db operator % (p2 x, p2 y) { return x.x * y.x + x.y * y.y; }
25 | int half(p2 x) { return x.y < 0 || (x.y == 0 && x.x <= 0); }
26 | int half(p2 x) { return x.y < -eps || (std::fabs(x.y) < eps && x.x < eps); }
27 | bool cmp(p2 a, p2 b) { return half(a) == half(b) ? a * b > 0 : half(b); }
28 | bool cmp_eq(p2 A, p2 B) { return half(A) == half(B) && eq(A * B, 0); }
29 | // 判断 A, B, C 三个向量是否是逆时针顺序
30 | // 如果是, 返回 1
31 | // 如果 (A, B), (C, B) 同方向共线, 返回 -1
32 | // 如果是顺时针, 返回 0
33 | bool cmp_ct(p2 A, p2 B, p2 C) {
34 | | if(cmp_eq(A, B)) return -1;
35 | | if(cmp_eq(C, B)) return -1;
36 | | if(cmp(A, B)) {
37 | | | return cmp(B, C) || cmp(C, A);
38 | | } else {
39 | | | return cmp(B, C) && cmp(C, A);
40 | | }
41 | }
42 | // 凸包 DP
43 | struct pr { int i, j; p2 get() const { return a[j] - a[i]; } };
44 | bool cmpseg(pr x, pr y) {

```

```

45 | p2 A = x.get(), B = y.get();
46 | if(!cmp(A, B) && !cmp(B, A)) return a[x.i] % A < a[y.i] % A;
47 | return cmp(A, B);
48 | }

```

6.2 三角形各心

```

1 p2 bary(p2 a, p2 b, p2 c, db A, db B, db C) {
2 | return (a * A + b * B + c * C) / (A + B + C);
3 | }
4 p2 centroid(p2 A, p2 B, p2 C) {
5 | return bary(A, B, C, 1, 1, 1);
6 | } // 重心
7 p2 circumcenter(p2 A, p2 B, p2 C) {
8 | db a = (B - C).norm(), b = (C - A).norm(), c = (A - B).norm();
9 | return bary(A, B, C, a*(b+c-a), b*(c+a-b), c*(a+b-c));
10 | } // 外心, 外接圆圆心, 三边中垂线的交点
11 p2 incenter(p2 A, p2 B, p2 C) {
12 | return bary(A, B, C, (B-C).abs(), (A-C).abs(), (A-B).abs());
13 | } // 内心, 内接圆圆心, 三角角平分线的交点
14 p2 orthocenter(p2 A, p2 B, p2 C) {
15 | db a = (B - C).norm(), b = (C - A).norm(), c = (A - B).norm();
16 | return bary(A, B, C, (a+b-c)*(c+a-b), (b+c-a)*(a+b-c), (c+a-b)*(b+c-a));
17 | } // 垂心, 三条高线所在的交点
18 p2 excenter(p2 A, p2 B, p2 C) {
19 | db a = (B - C).abs(), b = (A - C).abs(), c = (A - B).abs();
20 | return bary(A, B, C, -a, b, c);
21 | } // return bary(A, B, C, a, -b, c);
22 | } // return bary(A, B, C, a, b, -c);
23 | } // 旁心 一个内角的平分线和其他两个内角的外角平分线的交点

```

6.3 直线半平面

```

1 struct line : p2 {
2 | db z;
3 | // a * x + b * y + c (= or >) 0
4 | line() = default;
5 | line(db a, db b, db c) : p2{a, b}, z(c) {}
6 | line(p2 a, p2 b) : p2(r90(b - a)), z(a * b) {} //左侧 > 0
7 | db operator()(p2 a) const { return a % p2(*this) + z; }
8 | line perp() const { return {y, -x, 0}; } // 垂直
9 | line para(p2 o) { return {x, y, z - (*this)(o)}; } // 平行
10 | };
11 p2 operator & (line x, line y) {
12 | return p2{p2{x.z, x.y} * p2{y.z, y.y}, p2{x.x, x.z} * p2{y.x, y.z}} /
13 | -(p2(x) * p2(y));
14 | } // 注意此处精度误差较大, 以及 res.y 需要较高精度
15 p2 proj(p2 x, line l){return x - p2(l) * (l(x) / l.norm());} //投影
16 p2 refl(p2 x, line l){return x - p2(l) * (l(x) / l.norm()) * 2;} //对称
17 db dist(line l, p2 x={0, 0}){return l(x) / l.abs();} //有向点到线距离
18 bool is_para(line x, line y){return eq(p2(x) * p2(y), 0);} //判断线平行
19 bool is_perp(line x, line y){return eq(p2(x) % p2(y), 0);} //判断线垂直
20 bool online(p2 x, line l) { return eq(l(x), 0); } // 判断点在线上

```

```

21 int ccw(p2 a, p2 b, p2 c) {
22 | int sign = sgn((b - a) * (c - a));
23 | if(sign == 0) {
24 | | if(sgn((b - a) % (c - a)) == -1) return 2;
25 | | if((c - a).norm() > (b - a).norm() + eps) return -2;
26 | }
27 | return sign;
28 | }
29 db det(line a, line b, line c) {
30 | p2 A = a, B = b, C = c;
31 | return c.z * (A * B) + a.z * (B * C) + b.z * (C * A);
32 | }
33 db check(line a, line b, line c) { // sgn same as c(a & b), 0 if error
34 | return sgn(det(a, b, c)) * sgn(p2(a) * p2(b));
35 | }
36 bool paraS(line a, line b) { // 射线同向
37 | return is_para(a, b) && p2(a) % p2(b) > 0;
38 | }

```

6.4 半平面交

```

1 std::vector<p2> HPI(std::vector<line> vs) {
2 | auto cmp = [](line a, line b) {
3 | | if(paraS(a, b)) return dist(a) < dist(b);
4 | | return ::cmp(p2(a), p2(b));
5 | };
6 | sort(vs.begin(), vs.end(), cmp);
7 | int ah = 0, at = 0, n = size(vs);
8 | std::vector<line> deq(n + 1);
9 | std::vector<p2> ans(n);
10 | deq[0] = vs[0];
11 | for(int i = 1; i <= n; ++i) {
12 | | line o = i < n ? vs[i] : deq[ah];
13 | | if(paraS(vs[i - 1], o)) continue;
14 | | for(; ah < at && check(deq[at - 1], deq[at], o) < 0;) -- at; //maybe <=
15 | | if(i != n) for(; ah < at && check(deq[ah], deq[ah + 1], o) < 0;) ++ ah;
16 | | if(!is_para(o, deq[at])) {
17 | | | ans[at] = o & deq[at];
18 | | | deq[++at] = o;
19 | | }
20 | }
21 | if(at - ah <= 2) return {};
22 | return {ans.begin() + ah, ans.begin() + at};
23 | }

```

6.5 线段

```

1 struct seg {
2 | p2 x, y;
3 | seg() {}
4 | seg(const p2 & A, const p2 & B) : x(A), y(B) {}
5 | bool onseg(const p2 & o) const {
6 | | return (o - x) % (o - y) < eps && std::fabs((o - x) * (o - y)) < eps;
7 | }

```

```

8 };
9 db dist(const seg & o, const p2 & x) {
10 | if((o.x - o.y) % (x - o.y) <= eps) return (x - o.y).abs();
11 | if((o.y - o.x) % (x - o.x) <= eps) return (x - o.x).abs();
12 | return fabs((o.x - x) * (o.y - x) / (o.x - o.y).abs());
13 }
14 bool is_isc(const seg & x, const seg & y) {
15 | return
16 | | ccw(x.x, x.y, y.x) * ccw(x.x, x.y, y.y) <= 0 &&
17 | | ccw(y.x, y.y, x.x) * ccw(y.x, y.y, x.y) <= 0;
18 }
19 db dist(const seg & x, const seg & y) {
20 | if(is_isc(x, y)) return 0;
21 | return std::min({dist(y, x.x), dist(y, x.y), dist(x, y.x), dist(x, y.y)});
22 }

```

6.6 简单多边形

```

1 using polygon = std::vector<p2>;
2 // counter-clockwise
3 db area(const polygon & x) {
4 | db res = 0;
5 | for(int i = 2; i < (int) x.size(); ++i) {
6 | | res += (x[i - 1] - x[0]) * (x[i] - x[0]);
7 | }
8 | return res / 2;
9 }
10 bool is_convex(const polygon & x, bool strict = 1) {
11 | // warning, maybe wrong
12 | const db z = strict ? eps : -eps;
13 | for(int i = 2; i < (int) x.size() + 2; ++i) {
14 | | if((x[(i - 1) % x.size()] - x[i - 2]) * (x[i % x.size()] - x[i - 2]) <
15 | | z) return 0;
16 | }
17 | return 1;
18 }
19 int contain(const std::vector<p2> & a, p2 o) { // 简单多边形包含判定
20 | bool in = 0;
21 | for(int i = 0; i < (int) a.size(); ++i) {
22 | | p2 x = a[i] - o, y = a[(i + 1) % a.size()] - o;
23 | | if(x.y > y.y) std::swap(x, y);
24 | | if(x.y <= eps && y.y > eps && x * y < -eps) in ^= 1;
25 | | if(std::fabs(x * y) < eps && x % y < eps) return 2; // 在线段上, 看情况改
26 | }
27 | return in;

```

6.7 简单多边形三角剖分

```

1 std::vector<std::array<int, 3>> triangulate(std::vector<p2> a) {
2 | int n = a.size();
3 | std::vector<int> prev(n), next(n), ear(n);
4 | auto in = [](p2 p, p2 a, p2 b, p2 c) {

```

```

5 | | return area(p, a, b) >= -eps && area(p, b, c) >= -eps && area(p, c, a)
6 | | >= -eps;
7 | };
8 | auto is_ear = [&](int x) {
9 | | if(area(a[prev[x]], a[x], a[next[x]]) <= eps) return 0;
10 | | for(int i = next[next[x]]; i != prev[x]; i = next[i])
11 | | | if(in(a[i], a[prev[x]], a[x], a[next[x]]))
12 | | | return 0;
13 | | return 1;
14 | };
15 | for(int i = 0; i < n; ++i) {
16 | | next[i] = (i + 1) % n, prev[(i + 1) % n] = i;
17 | }
18 | for(int i = 0; i < n; ++i) {
19 | | ear[i] = is_ear(i);
20 | }
21 | std::vector<std::array<int, 3>> ans(n - 2);
22 | for(int ec = 0, cur = 0; ec < n - 2; ++ec) {
23 | | for(!ear[cur];) cur = next[cur];
24 | | int a = prev[cur], b = cur, c = next[cur];
25 | | ans[ec] = {a, b, c};
26 | | next[a] = c, prev[c] = a;
27 | | ear[a] = is_ear(a), ear[c] = is_ear(c);
28 | | cur = next[cur];
29 | }
30 | return ans;
31 } // 输入逆时针

```

6.8 线段 in 多边形

```

1 bool contains(p2 x, p2 y, const std::vector<p2> & a) {
2 | using pr = std::pair<double, int>;
3 | std::vector<pr> e = {pr(-inf, 0), pr(inf, 0)};
4 | p2 t = y - x;
5 | auto f = [&](p2 a, p2 b, p2 c, p2 d) {
6 | | return (b - a).abs() * ((c - a) * (d - c)) / ((b - a) * (d - c));
7 | };
8 | for(int i = 0; i < (int) a.size(); ++i) {
9 | | p2 u = a[i], v = a[(i + 1) % a.size()];
10 | | int a = sgn(t * (u - x));
11 | | int b = sgn(t * (v - x));
12 | | if(a != b) e.emplace_back(f(x, y, u, v), b - a);
13 | }
14 | sort(e.begin(), e.end());
15 | int sum = 0; db R = t.abs();
16 | for(int i = 0; i + 1 < (int) e.size(); ++i) {
17 | | sum += e[i].second;
18 | | if(sum == 0 && std::max(e[i].first, 0.) + eps < std::min(e[i +
19 | | 1].first, R)) {
20 | | | return 0;
21 | | }
22 | }
23 | return 1;

```

6.9 图形求交

```

1 struct circle : p2 { db r; };
2 std::vector<p2> operator & (circle o, line l) {
3     p2 v = l, Rv = r90(v); db L = l.abs();
4     db d = l(p2(o)) / L, x = o.r * o.r - d * d;
5     if(x < -eps) return {};
6     x = std::sqrt(x * sgn(x));
7     p2 z = p2(o) - v * (d / L), p = Rv * (x / L);
8     return {z + p, z - p};
9 } // l 如果是构造函数给出, 那么返回交点按射线顺序
10 std::vector<p2> operator & (circle o, seg s) {
11     std::vector<p2> b;
12     for(p2 x : (o & s.to_l()))
13         if(s.onseg(x)) b.push_back(x);
14     return b;
15 }
16 std::vector<p2> operator & (circle o0, circle o1) {
17     p2 tmp = (p2(o1) - p2(o0)) * 2.;
18     return o0 & line(tmp.x, tmp.y, o1.r * o1.r - o0.r * o0.r + o0.norm() -
19         o1.norm());
20 }
21 std::vector<p2> tang(circle o, p2 x) {
22     db d = (x - p2(o)).abs();
23     if(d <= o.r + eps) return {};
24     return o & circle{x, sqrt(d * d - o.r * o.r)};
25 }
26 // 三角形 (o, a, b) 和圆 o 的交的有向面积 * 2
27 db intersect(circle o, p2 a, p2 b) {
28     a = a - p2(o), b = b - p2(o); o.x = o.y = 0;
29     int va = a.abs() <= o.r + eps;
30     int vb = b.abs() <= o.r + eps;
31     if(va && vb) return a * b;
32     auto v = o & seg{a, b}; // 注意这里, 有必要改一下 onseg, 去掉平行判定
33     if(v.empty()) return arg(a, b) * o.r * o.r;
34     db sum = 0;
35     sum += va ? a * v[0] : arg(a, v[0]) * o.r * o.r;
36     sum += vb ? v.back() * b : arg(v.back(), b) * o.r * o.r;
37     if(v.size() > 1) sum += v[0] * v[1];
38     return sum;
39 }
40 // 有向弓形面积 * 2, arg 不能改
41 db csegS(circle o, p2 a, p2 b) {
42     a = a - p2(o);
43     b = b - p2(o);
44     db d = b.arg() - a.arg();
45     if(d < 0) d += pi * 2;
46     return d * o.r * o.r - a * b;
47 }
48 // 两圆交的面积 * 2
49 db intersect(circle o0, circle o1) {
50     if(o0.r > o1.r) std::swap(o0, o1);
51     db d = (p2(o0) - p2(o1)).abs();
52     if(d <= (o1.r - o0.r) + eps) return 2 * pi * o0.r * o0.r;
53     if(d >= o1.r + o0.r - eps) return 0;

```

```

53 | auto v = o0 & o1;
54 | return csegS(o0, v[1], v[0]) + csegS(o1, v[0], v[1]);
55 }

```

6.10 凸包

结果为逆时针。

```

1 db cross(p2 x, p2 y, p2 z) { return (y.x - x.x) * (z.y - x.y) - (y.y - x.y) *
2     (z.x - x.x); }
3 std::vector<p2> gethull(std::vector<p2> o) {
4     rgs::sort(o, [](p2 x, p2 y) { return eq(x.x, y.x) ? x.y < y.y : x.x < y.x;
5     });
6     o.erase(unique(o.begin(), o.end(), [](p2 x, p2 y) {
7         return eq(x.x, y.x) && eq(x.y, y.y);
8     }), o.end());
9     std::vector<p2> s;
10    for(int i = 0; i < (int) o.size(); ++i) {
11        for(; s.size() >= 2 && cross(s.rbegin()[1], s.back(), o[i]) <= eps;)
12            s.pop_back();
13        s.push_back(o[i]);
14    }
15    for(int i = o.size() - 2, t = s.size(); i >= 0; --i) {
16        for(; s.size() > t && cross(s.rbegin()[1], s.back(), o[i]) <= eps;)
17            s.pop_back();
18        s.push_back(o[i]);
19    }
20    if(s.size() > 1) s.pop_back();
21    return s;
22 } // 把两个 eps 改成 -eps 可求出所有在凸包上的点
23 int findmin(std::vector<p2> & a, auto cmp) {
24     int l = 0, r = a.size() - 1, d = 1;
25     if(cmp(a.back(), a[0])) std::swap(l, r), d = -1;
26     for(; (r - l) * d > 1;) {
27         int mid = (l + r) >> 1;
28         if(cmp(a[mid], a[mid - d]) && cmp(a[mid], a[l])) {
29             l = mid;
30         } else {
31             r = mid;
32         }
33     }
34     return l;
35 } // cmp is less, and a.size()>0 plz
36 int contains(std::vector<p2> & a, p2 x) {
37     auto it = lower_bound(a.begin() + 2, a.end(), x, [](p2 x, p2 y) {
38         return cross(a[0], x, y) > 0;
39     });
40     ll c0 = cross(it[-1], *it, x), c1 = cross(a[0], a[1], x);
41     if(it != a.end() && c0 >= 0 && c1 >= 0) {
42         return c0 > 0 && c1 > 0 && cross(a.back(), a[0], x) > 0 ? IN : ON;
43     } else {
44         return 0;
45     }
46 } // a.size()>2 plz

```


6.11 上凸壳

结果显然为顺时针。

```
1 std::vector<p2> gethull(std::vector<p2> o) {
2   sort(o.begin(), o.end(), [](p2 x, p2 y) {
3     if(x.x == y.x) {
4       return x.y > y.y; // gt ⇒ lt
5     } else {
6       return x.x < y.x;
7     }
8   });
9   std::vector<p2> stack;
10  for(p2 x : o) {
11    if(stack.size() && stack.back().x == x.x) {
12      continue;
13    }
14    for(;;stack.size() >= 2 && cross(stack.rbegin()[1], stack.back(), x) >=
15      0;) { // gt ⇒ lt
16      stack.pop_back();
17    }
18    stack.push_back(x);
19  }
20  return stack;
}
```

6.12 最小圆覆盖

```
1 struct circle : p2 { db r; };
2 circle incircle(p2 a, p2 b, p2 c) {
3   db A = (b - c).abs(), B = (c - a).abs(), C = (a - b).abs();
4   return {(a * A + b * B + c * C) / (A + B + C), fabs((b - a) * (c - a)) /
5     (A + B + C)};
6 } // 三点确定内心, 不是最小圆覆盖内容
7 circle circumcircle(p2 a, p2 b, p2 c) {
8   p2 bc = c - b, ca = a - c, ab = b - a;
9   p2 o = (b + c - r90(bc) * (ca % ab) / (ca * ab)) / 2;
10  return {o, (a - o).abs()};
11 } // 三点确定外接圆
12 circle cir(p2 a, p2 b) { // 根据直径生成圆
13   return {(a + b) / 2, (a - b).abs() / 2};
14 }
15 bool in(circle x, p2 y) { return (p2(x) - y).abs() <= x.r + eps; }
16 circle mincircle(std::vector<p2> a) { // 最小圆覆盖, 需要 shuffle
17   circle o = {a[0], 0};
18   int n = a.size();
19   for(int i = 1; i < n; ++i) {
20     if(in(o, a[i])) continue;
21     o = cir(a[0], a[i]);
22     for(int j = 1; j < i; ++j) {
23       if(in(o, a[j])) continue;
24       o = cir(a[j], a[i]);
25       for(int k = 0; k < j; ++k) {
26         if(in(o, a[k])) continue;
27       }
28     }
29   }
30 }
```

```
26 | | | | o = circumcircle(a[i], a[j], a[k]);
27 | | | | }
28 | | | }
29 | | }
30 | return o;
31 }
```

6.13 最近点对

```
1 db mindist(std::vector<p2> a) {
2   db ans = 1e18;
3   sort(a.begin(), a.end(), [](p2 x, p2 y) { return x.x < y.x; });
4   ans = (a[0] - a[1]).abs();
5   auto solve = [&](auto s, int l, int r) {
6     if(l + 1 == r) return;
7     int mid = (l + r) >> 1;
8     db mx = a[mid].x;
9     s(s, l, mid), s(s, mid, r);
10    static std::vector<p2> b; b.clear();
11    inplace_merge(a.begin() + l, a.begin() + mid, a.begin() + r, [](p2 x,
12      p2 y) { return x.y < y.y; });
13    for(int i = l; i < r; ++i)
14      if(fabs(a[i].x - mx) <= ans) b.push_back(a[i]);
15    for(int i = 1; i < (int) b.size(); ++i)
16      for(int j = i - 1; j >= 0 && b[i].y <= b[j].y + ans; --j) ans =
17        std::min(ans, (b[i] - b[j]).abs());
18  };
19  solve(solve, 0, a.size());
20  return ans;
21 }
```

6.14 凸包直径

```
1 db convex_diameter(std::vector<p2> & o) {
2   int n = size(o);
3   db max = 0;
4   for(int i = 0, j = 1 % n; i < j; ++i) {
5     for(;;j = (j + 1) % n) {
6       max = std::max(max, (o[j] - o[i]).abs());
7       if((o[i + 1] - o[i]) * (o[(j + 1) % n] - o[j]) <= 0) break;
8     }
9   }
10  return max;
11 } // 凸包直径, 不能有三点共线
```

6.15 切凸包

```
1 std::vector<p2> cut(const std::vector<p2> & o, line l) {
2   std::vector<p2> res;
3   int n = size(o);
4   for(int i = 0; i < n; ++i) {
5     p2 a = o[i], b = o[(i + 1) % n];
6     if(sgn(l(a)) >= 0) res.push_back(a); // 注意 sgn 精度
7   }
8 }
```

```

7 | | if(sgn(l(a)) * sgn(l(b)) < 0) res.push_back(line(a, b) & l);
8 | }
9 | if(res.size() <= 2) return {};
10 | return res;
11 | } // 切凸包

```

6.16 V图

```

1 | std::vector<line> cut(const std::vector<line> & o, line l) {
2 |     std::vector<line> res;
3 |     int n = size(o);
4 |     for(int i = 0; i < n; ++i) {
5 |         line a = o[i], b = o[(i + 1) % n], c = o[(i + 2) % n];
6 |         int va = check(a, b, l), vb = check(b, c, l);
7 |         if(va > 0 || vb > 0 || (va == 0 && vb == 0)) {
8 |             res.push_back(b);
9 |         }
10 |         if(va >= 0 && vb < 0) {
11 |             res.push_back(l);
12 |         }
13 |     }
14 |     if(res.size() <= 2) return {};
15 |     return res;
16 | } // 切凸包
17 | line bisector(p2 a, p2 b) { return line(a.x - b.x, a.y - b.y, (b.norm() -
18 |     a.norm()) / 2); }
19 | line bisector(circle a, circle b) {
20 |     return line(a.x - b.x, a.y - b.y, (b.norm() - a.norm() + a.r * a.r - b.r *
21 |         b.r) / 2);
22 | } // a b 不能是包含关系, for power diagram
23 |
24 | std::vector<std::vector<line>> voronoi(std::vector<p2> p) {
25 |     int n = p.size();
26 |     auto b = p; shuffle(b.begin(), b.end(), gen);
27 |     const db V = 1e5; // 边框大小, 重要
28 |     std::vector<std::vector<line>> a(n, {
29 |         {V, 0, V * V}, {0, V, V * V},
30 |         {-V, 0, V * V}, {0, -V, V * V},
31 |     });
32 |     for(int i = 0; i < n; ++i) {
33 |         for(p2 x : b) if((x - p[i]).abs() > eps) {
34 |             a[i] = cut(a[i], bisector(p[i], x));
35 |         }
36 |     }
37 |     return a;
38 | }

```

6.17 Delaunay 三角剖分

```

1 | using i128 = __int128;
2 | using Q = struct Quad*;
3 | p2 arb(LLONG_MAX, LLONG_MAX);
4 | struct Quad {
5 |     Q rot, o; p2 p = arb; bool mark;

```

```

6 |     p2& F() { return r() -> p; }
7 |     Q& r() { return rot->rot; }
8 |     Q prev() { return rot->o->rot; }
9 |     Q next() { return r()->prev(); }
10 | } *H;
11 | ll cross(p2 a, p2 b, p2 c) {
12 |     return (b - a) * (c - a);
13 | }
14 | bool circ(p2 p, p2 a, p2 b, p2 c) { // p 是否在 a, b, c 外接圆中
15 |     i128 p2 = p.norm(), A = a.norm() - p2, B = b.norm() - p2, C = c.norm() -
16 |         p2;
17 |     a = a - p, b = b - p, c = c - p;
18 |     return (a * b) * C + (b * c) * A + (c * a) * B > 0;
19 | }
20 | Q link(p2 orig, p2 dest) {
21 |     Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
22 |     H = r -> o; r -> r() -> r() = r;
23 |     for(int i = 0; i < 4; ++i)
24 |         r = r -> rot, r -> p = arb, r -> o = i & 1 ? r : r -> r();
25 |     r -> p = orig, r -> F() = dest;
26 |     return r;
27 | }
28 | void splice(Q a, Q b) {
29 |     std::swap(a -> o -> rot -> o, b -> o -> rot -> o);
30 |     std::swap(a -> o, b -> o);
31 | }
32 | Q conn(Q a, Q b) {
33 |     Q q = link(a -> F(), b -> p);
34 |     splice(q, a -> next());
35 |     splice(q -> r(), b);
36 |     return q;
37 | }
38 | std::pair<Q, Q> rec(const std::vector<p2> & s) {
39 |     int N = size(s);
40 |     if(N <= 3) {
41 |         Q a = link(s[0], s[1]), b = link(s[1], s.back());
42 |         if(N == 2) return {a, a -> r()};
43 |         splice(a -> r(), b);
44 |         ll side = cross(s[0], s[1], s[2]);
45 |         Q c = side ? conn(b, a) : 0;
46 |         return {side < 0 ? c->r() : a, side < 0 ? c : b -> r()};
47 |     }
48 | #define H(e) e -> F(), e -> p
49 | #define valid(e) (cross(e->F(), H(base)) > 0)
50 |     int half = N / 2;
51 |     auto [ra, A] = rec({s.begin(), s.end() - half});
52 |     auto [B, rb] = rec({s.end() - half, s.end()});
53 |     while((cross(B -> p, H(A)) < 0 && (A = A -> next())) ||
54 |         ((cross(A -> p, H(B)) > 0 && (B = B -> r() -> o)));
55 |     Q base = conn(B -> r(), A);
56 |     if(A -> p == ra -> p) ra = base -> r();
57 |     if(B -> p == rb -> p) rb = base;
58 | #define DEL(e, init, dir) Q e = init -> dir; if(valid(e)) \
59 |     for(; circ(e -> dir -> F(), H(base), e -> F());) { \
60 |         Q t = e -> dir; \

```

```

60 | | splice(e, e -> prev()); \
61 | | splice(e -> r(), e -> r() -> prev()); \
62 | | e -> o = H, H = e, e = t; \
63 | }
64 | for(;;) {
65 | | DEL(LC, base -> r(), o);
66 | | DEL(RC, base, prev());
67 | | if(!valid(LC) && !valid(RC)) break;
68 | | if(!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
69 | | | base = conn(RC, base -> r());
70 | | else
71 | | | base = conn(base -> r(), LC -> r());
72 | }
73 | return {ra, rb};
74 | }
75 | std::vector<p2> triangulate(std::vector<p2> a) {
76 | | sort(a.begin(), a.end()); // unique
77 | | if((int)size(a) < 2) return {};
78 | | Q e = rec(a).first;
79 | | std::vector<Q> q = {e};
80 | | while(cross(e -> o -> F(), e -> F(), e -> p) < 0) e = e -> o;
81 | #define ADD { Q c = e; do { c -> mark = 1; a.push_back(c -> p); \
82 | | q.push_back(c -> r()), c = c -> next(); } while(c != e); }
83 | | ADD; a.clear();
84 | | for(int qi = 0; qi < (int) size(q);) if(!(e = q[qi++]) -> mark) ADD;
85 | | return a;
86 | } // 返回若干逆时针三角形 \{t[0][0], t[0][1], t[0][2], t[1][0], \dots\}

```

7 geometry3d

7.1 向量

```

1 | struct p3 {
2 | | db x, y, z;
3 | | db norm() const { return x * x + y * y + z * z; }
4 | | db abs() const { return std::sqrt(norm()); }
5 | };
6 | p3 operator + (p3 x, p3 y){ return {x.x + y.x, x.y + y.y, x.z + y.z}; }
7 | p3 operator - (p3 x, p3 y){ return {x.x - y.x, x.y - y.y, x.z - y.z}; }
8 | p3 operator * (p3 x, db y) { return {x.x * y, x.y * y, x.z * y}; }
9 | p3 operator / (p3 x, db y) { return {x.x / y, x.y / y, x.z / y}; }
10 | p3 operator * (p3 x, p3 y) { // 三维叉积需要更高的精度
11 | | return {
12 | | | x.y * y.z - x.z * y.y,
13 | | | x.z * y.x - x.x * y.z,
14 | | | x.x * y.y - x.y * y.x
15 | | };
16 | }
17 | db operator % (p3 x, p3 y) { return x.x * y.x + x.y * y.y + x.z * y.z; }
18 | p3 perpvec(p3 x) {
19 | | return fabs(x.x) > fabs(x.z) ? p3{ x.y, -x.x, 0 } : p3{ 0, -x.z, x.y };
20 | | // 找到一个与给定向量垂直的向量
21 | }
22 | db area(p3 a, p3 b, p3 c) { return ((b - a) * (c - a)).abs(); } // 三角形面积两
倍

```

```

22 | db volume(p3 d, p3 a, p3 b, p3 c) { // 四面体有向体积六倍
23 | | return (d - a) % ((b - a) * (c - a));
24 | }

```

7.2 平面

```

1 | struct plane {
2 | | p3 n; db d; // n dot x = d
3 | | plane() {}
4 | | plane(p3 a, p3 b, p3 c) : n((c - a) * (b - a)) { d = n % a; }
5 | | db side(p3 x) const { return n % x - d; }
6 | | db dist(p3 w) const { return side(w) / n.abs(); }
7 | | p3 proj(p3 w) const { return w - n * (side(w) / n.norm()); }
8 | };

```

7.3 直线

```

1 | struct line3 {
2 | | p3 d, o; // kd + o
3 | | line3() {}
4 | | line3(p3 p, p3 q) : d(q - p), o(p) {}
5 | | line3(plane p1, plane p2) : d(p1.n * p2.n) { // 平面交出直线
6 | | | o = (p2.n * p1.d - p1.n * p2.d) * d / d.norm();
7 | | }
8 | | db dist(p3 p) const { return (d * (p - o)).abs() / d.abs(); }
9 | | p3 proj(p3 p) const { return o + d * (d % (p - o)) / d.norm(); } // 投影
10 | | p3 relf(p3 p) const { return proj(p) * 2 - p; } // 对称
11 | | p3 operator & (const plane & p) const { // 线与平面交
12 | | | return o - d * p.side(o) / (p.n % d);
13 | | }
14 | };
15 | db dist(line3 l1, line3 l2) {
16 | | p3 n = l1.d * l2.d;
17 | | if(n.abs() < eps) return l1.dist(l2.o);
18 | | return abs((l2.o - l1.o) % n) / n.abs();
19 | }
20 | p3 closestOnL1(line3 l1, line3 l2) {
21 | | p3 n2 = l2.d * (l1.d * l2.d);
22 | | return l1.o + l1.d * ((l2.o - l1.o) % n2) / (l1.d % n2);
23 | }

```

```

24 | bool ispara(plane p1, plane p2){return(p1.n * p2.n).abs() < eps;} // 判断是否平行
25 | bool ispara(line3 p1, line3 p2){return(p1.d * p2.d).abs() < eps;} // 判断是否平行
26 | bool isperp(plane p1, plane p2){return fabs(p1.n % p2.n) < eps;} // 判断是否垂直
27 | bool isperp(line3 p1, line3 p2){return fabs(p1.d % p2.d) < eps;} // 判断是否垂直
28 | line3 perpthrough(plane p, p3 o){return line3(o, o + p.n);} // 过平面一点做垂线

```

7.4 凸包

```

1 | const int N = 2005;
2 | struct face { int a[3]; plane p; };
3 | int vis[N][N];
4 | std::vector<face> f;
5 | void convex3d(const std::vector<p3> & a) { // need to deal coplane

```

```

6 | if(a.size() < 3) return ;
7 | auto getface = [&](int i, int j, int k) -> face { return {{i, j, k},
  | plane(a[i], a[j], a[k])}}; };
8 | f = {getface(0, 1, 2), getface(0, 2, 1)};
9 | std::vector<face> tmp[2];
10 | for(int i = 3; i < (int) a.size(); ++i) {
11 |     for(auto x : f) {
12 |         if(x.p.dist(a[i]) < -eps) {
13 |             tmp -> push_back(x);
14 |         } else {
15 |             tmp[1].push_back(x);
16 |             for(int t : {0, 1, 2}) vis[x.a[t]][x.a[(t + 1) % 3]] = i;
17 |         }
18 |     }
19 |     for(auto x : tmp[1]) {
20 |         for(int t : {0, 1, 2}) {
21 |             if(vis[x.a[t]][x.a[(t + 1) % 3]] == i && vis[x.a[(t + 1) % 3]]
  | [x.a[t]] != i)
22 |                 tmp[0].push_back(getface(x.a[t], x.a[(t + 1) % 3], i));
23 |         }
24 |     }
25 |     f = tmp[0]; tmp[0].clear(); tmp[1].clear();
26 | }
27 | for(int i = 0; i < (int)a.size(); ++i) memset(vis[i], 0, a.size() << 2);
28 | }

```

8 Misc

8.1 Pragma

```

1 | #pragma GCC optimize("Ofast")
2 | #pragma GCC optimize("unroll-loops")
3 | #pragma GCC target("sse,sse2,sse3,ssse3,sse4,popcnt,abm,mmx,avx,avx2")
4 | #pragma pack(1) // default=8

```

8.2 Barrett

```

1 | struct DIV {
2 |     u64 x;
3 |     void init(u64 v) { x = -1ull / v + 1; }
4 | }; // 带误差版本 x = -1ull/v;
5 | // ret=ans while x*(y-1)<2^64, ans-1<=ret<=ans while x<2^64
6 | u64 operator / (const u64 & x, const DIV & y) {
7 |     return (u128) x * y.x >> 64;
8 | }

```

8.3 快速整除判定

要求 p 是奇数。如果是 (u)int128, 求逆循环次数要改成 6。

```

1 | struct Divcheck {
2 |     u64 inv, l;

```

```

3 | Divcheck(u64 p) {
4 |     inv = p, l = -1ull / p;
5 |     for(int j = 0; j < 5; ++j) inv *= 2 - inv * p;
6 | }
7 | bool check(u64 x) const {
8 |     return x * inv <= l;
9 | }
10 | };
11 | struct Divcheckll {
12 |     u64 inv; ll l, r;
13 |     Divcheckll(ll p) {
14 |         inv = p, l = LLONG_MIN / p, r = LLONG_MAX / p;
15 |         for(int j = 0; j < 5; ++j) inv *= 2 - inv * p;
16 |     }
17 |     bool check(ll x) const {
18 |         return x * inv, l <= x && x <= r;
19 |     }
20 | };

```

8.4 Montgomery

要求 p 是奇数, 不要超过 2^{62} 。reduce 计算 $\frac{x}{2^{64}} \pmod{p}$, 如果输入在 $[0, p \times 2^{64})$ 内, 返回值在 $[0, 2p)$ 内。

```

1 | using u64 = unsigned long long;
2 | using u128 = __uint128_t;
3 | struct mont {
4 |     u64 p, pr, r;
5 |     mont(u64 p) : p(p) {
6 |         pr = p, r = (-u128(p)) % p;
7 |         for(int i = 0; i < 6; ++i) pr *= 2 - p * pr;
8 |     }
9 |     u64 reduce(u128 x) const {
10 |         u64 c = x, d = x >> 64;
11 |         return d + p - u64(u128(c * pr) * p >> 64);
12 |     }
13 |     u64 raw(u64 x) const { return reduce(x * u128(r)); }
14 |     u64 val(u64 x) const { return x = reduce(x), x >= p ? x - p : x; }
15 | };

```

8.5 Pragma

```

1 | #pragma GCC optimize("Ofast")
2 | #pragma GCC optimize("unroll-loops")
3 | #pragma GCC target("sse,sse2,sse3,ssse3,sse4,popcnt,abm,mmx,avx,avx2")
4 | #pragma pack(1) // default=8

```

8.6 LCS

```

1 | int lim;
2 | struct bitset {
3 |     static const int B = 63;
4 |     u64 a[N / B + 1];

```

```

5 | void set(int p) { a[p / B] |= 1ull << (p % B); }
6 | bool test(int p) { return a[p / B] >> (p % B) & 1; }
7 | void run(const bitset & o) {
8 |     u64 c = 1;
9 |     for(int i = 0; i < lim; ++i) {
10 |         u64 x = a[i], y = x | o.a[i];
11 |         x += x + c + (~y & (1ull << 63) - 1);
12 |         a[i] = x & y, c = x >> 63;
13 |     }
14 | }
15 | } dp;

```

8.7 日期公式

getday 返回自 1/1/1 起到 $y/m/d$ 的天数, 1/1/1 是星期一, 所以模 7 可以得算是周几。

date 是 getday 的逆。

```

1 // Mon = 0, ... % 7
2 // days since 1/1/1
3 int getday(int y, int m, int d) {
4 |     if(m < 3) -- y, m += 12;
5 |     return (365 * y + y / 4 - y / 100 + y / 400 + (153 * (m - 3) + 2) / 5 + d
6 |         - 307);
7 }
8 void date(int n, int & y, int & m, int & d) {
9 |     n += 429 + ((4 * n + 1227) / 146097 + 1) * 3 / 4;
10 |     y = (4 * n - 489) / 1461;
11 |     n -= y * 1461 / 4;
12 |     m = (5 * n - 1) / 153;
13 |     d = n - m * 153 / 5;
14 |     if(--m > 12) m -= 12, ++y;
15 }

```

8.8 Xorshift

```

1 u64 xorshift(u64 x) { x ^= x << 13; x ^= x >> 7; x ^= x << 17; return x; }
2 u32 xorshift(u32 x) { x ^= x << 13; x ^= x >> 17; x ^= x << 5; return x; }

```

8.9 distributions

8.10 python

```

1 import sys
2 sys.set_int_max_str_digits(10**9) # for large integer string conversion
3 sys.stdout.flush() # IO 交互
4 import math
5 print(math.comb(10, 5), math.factorial(10))
6 print(math.gcd(3, 6, 9), math.gcd(16, 12))
7 print(math.lcm(4, 6, 9), math.lcm(4, 8))
8 print(math.isqrt(5))
9 from fractions import Fraction
10 a=Fraction(math.e).limit_denominator(10) # 返回分母不超过 10 的最近分数,
    ↪ default=1e6

```

```

11 u,v=a.as_integer_ratio()
12 u,v=a.numerator,a.denominator # 和上面一样
13 print(u,'/',v)
14 from decimal import*
15 getcontext().prec = 100 #default=28
16 a, b = Decimal(114), Decimal('514')
17 print(a/b,(a/b)**3)
18 import sys
19 input=lambda:sys.stdin.readline().rstrip() # 快读
20 print('haha', flush=True) # 交互用
21 sys.stdout.flush() # 或者用这个

```

8.11 浮点数精度

float : 1 bit sign, 8 bit exponent, 23 bit mantissa
double : 1 bit sign, 11 bit exponent, 52 bit mantissa
long double : 1 bit sign, 15 bit exponent, 64 bit mantissa

9 配置

9.1 vimrc

```

1 set si ci ts=4 sw=4 nu cino=j1 backup undofile
2 syntax on
3 map<F9> <ESC>:!g++ % -o %< -std=gnu++20 -g -Wall -Dzqj<CR>
4 map<F10> <ESC>:!./%<<CR>
5 map<F4> <ESC>:!gdb %<<CR>

```

9.2 bashrc

```

1 ulimit -s 1048576
2 ulimit -v 1048576

```

9.3 对拍

需要 chmod +x

```

1 while true; do
2 | ./gen > 1.in
3 | ./naive < 1.in > std.out
4 | ./a < 1.in > 1.out
5 | if diff 1.out std.out; then
6 |     echo ac
7 | else
8 |     echo wa
9 |     break
10 | fi
11 done

```

9.4 编译参数

-D_GLIBCXX_DEBUG : STL debug mode
-fsanitize=address : 内存错误检查
-fsanitize=undefined : UB 检查

9.5 随机素数

988153981 947090107248063517
905938739 937684197942221999
939136603 926543231406238423
968054167 950497126685825867
912657901 908159627021209279

9.6 常数表

n	$\log_{10} n$	$n!$	$C(n, n/2)$	$\text{LCM}(1 \dots n)$	P_n	
2	0.30102999	2	2	2	2	
3	0.47712125	6	3	6	3	
4	0.60205999	24	6	12	5	
5	0.69897000	120	10	60	7	
6	0.77815125	720	20	60	11	
7	0.84509804	5040	35	420	15	
8	0.90308998	40320	70	840	22	
9	0.95424251	362880	126	2520	30	
10	1	3628800	252	2520	42	
11	1.04139269	39916800	462	27720	56	
12	1.07918125	479001600	924	27720	77	
15	1.17609126	1.31e12	6435	360360	176	
20	1.30103000	2.43e18	184756	232792560	627	
25	1.39794001	1.55e25	5200300	26771144400	1958	
30	1.47712125	2.65e32	155117520	1.444e14	5604	
P_n	37338 ₄₀	204226 ₅₀	966467 ₆₀	190569292 ₁₀₀	1e9 ₁₁₄	
$n \leq$	10	100	1e3	1e4	1e5	1e6
$\max \omega(n)$	2	3	4	5	6	7
$\max d(n)$	4	12	32	64	128	240
$\pi(n)$	4	25	168	1229	9592	78498
$n \leq$	1e7	1e8	1e9	1e10	1e11	1e12
$\max \omega(n)$	8	8	9	10	10	11
$\max d(n)$	448	768	1344	2304	4032	6720
$\pi(n)$	664579	5761455	5.08e7	4.55e8	4.12e9	3.7e10
$n \leq$	1e13	1e14	1e15	1e16	1e17	1e18
$\max \omega(n)$	12	12	13	13	14	15
$\max d(n)$	10752	17280	26880	41472	64512	103680
$\pi(n)$	Prime number theorem: $\pi(x) \sim x/\log(x)$					

10 注意事项

10.1 测试项目

pbds tree, float128, int128, long double, submit 命令, printfile, MLE ?= RE, pragma, axv2, python,
测试代码长度限制, 尝试触发 NO-OUTPUT, OUTPUT-LIMIT, RUN-ERROR

10.2 bugs

看数据范围（多测总和），变量 shadow, 清空, long long, 数组大小, 模数, MLE?, 对拍记得看输出在不在变, 输出格式, inf 开小, 答案初值, STL 重构导致引用失效, 极端情况 (n=1)

11 tables

11.1 导数积分

$$\begin{array}{lll}
 (\frac{u}{v})' = \frac{u'v - uv'}{v^2} & (\arctan x)' = \frac{1}{1+x^2} & (\operatorname{arcsinh} x)' = \frac{1}{\sqrt{1+x^2}} \\
 (a^x)' = (\ln a)a^x & (\operatorname{arccot} x)' = -\frac{1}{1+x^2} & (\operatorname{arccosh} x)' = \frac{1}{\sqrt{x^2-1}} \\
 (\tan x)' = \sec^2 x & (\operatorname{arccsc} x)' = -\frac{1}{x\sqrt{1-x^2}} & (\operatorname{arctanh} x)' = \frac{1}{1-x^2} \\
 (\cot x)' = \csc^2 x & (\operatorname{arcsec} x)' = \frac{1}{x\sqrt{1-x^2}} & (\operatorname{arcoth} x)' = \frac{1}{x^2-1} \\
 (\sec x)' = \tan x \sec x & (\tanh x)' = \operatorname{sech}^2 x & (\operatorname{arcsch} x)' = -\frac{1}{x\sqrt{1+x^2}} \\
 (\csc x)' = -\cot x \csc x & (\coth x)' = -\operatorname{csch}^2 x & (\operatorname{arcsech} x)' = -\frac{1}{x\sqrt{1-x^2}} \\
 (\arcsin x)' = \frac{1}{\sqrt{1-x^2}} & (\operatorname{sech} x)' = -\operatorname{sech} x \tanh x & \\
 (\arccos x)' = -\frac{1}{\sqrt{1-x^2}} & (\operatorname{csch} x)' = -\operatorname{csch} x \coth x &
 \end{array}$$

$$ax^2 + bx + c (a > 0)$$

$$\begin{aligned}
 1. \int \frac{dx}{ax^2+bx+c} &= \begin{cases} \frac{2}{\sqrt{4ac-b^2}} \arctan \frac{2ax+b}{\sqrt{4ac-b^2}} + C & (b^2 < 4ac) \\ \frac{1}{\sqrt{b^2-4ac}} \ln \left| \frac{2ax+b-\sqrt{b^2-4ac}}{2ax+b+\sqrt{b^2-4ac}} \right| + C & (b^2 > 4ac) \end{cases} \\
 2. \int \frac{x}{ax^2+bx+c} dx &= \frac{1}{2a} \ln |ax^2 + bx + c| - \frac{b}{2a} \int \frac{dx}{ax^2+bx+c}
 \end{aligned}$$

$$\sqrt{\pm ax^2 + bx + c} (a > 0)$$

$$\begin{aligned}
 1. \int \frac{dx}{\sqrt{ax^2+bx+c}} &= \frac{1}{\sqrt{a}} \ln |2ax + b + 2\sqrt{a}\sqrt{ax^2 + bx + c}| + C \\
 2. \int \sqrt{ax^2 + bx + c} dx &= \frac{2ax+b}{4a} \sqrt{ax^2 + bx + c} + \frac{4ac-b^2}{8\sqrt{a^3}} \ln |2ax + b + 2\sqrt{a}\sqrt{ax^2 + bx + c}| + C \\
 3. \int \frac{x}{\sqrt{ax^2+bx+c}} dx &= \frac{1}{a} \sqrt{ax^2 + bx + c} - \frac{b}{2\sqrt{a^3}} \ln |2ax + b + 2\sqrt{a}\sqrt{ax^2 + bx + c}| + C \\
 4. \int \frac{dx}{\sqrt{c+bx-ax^2}} &= -\frac{1}{\sqrt{a}} \arcsin \frac{2ax-b}{\sqrt{b^2+4ac}} + C \\
 5. \int \sqrt{c+bx-ax^2} dx &= \frac{2ax-b}{4a} \sqrt{c+bx-ax^2} + \frac{b^2+4ac}{8\sqrt{a^3}} \arcsin \frac{2ax-b}{\sqrt{b^2+4ac}} + C \\
 6. \int \frac{x}{\sqrt{c+bx-ax^2}} dx &= -\frac{1}{a} \sqrt{c+bx-ax^2} + \frac{b}{2\sqrt{a^3}} \arcsin \frac{2ax-b}{\sqrt{b^2+4ac}} + C
 \end{aligned}$$

$$\sqrt{\pm \frac{x-a}{x-b}} \text{ 或 } \sqrt{(x-a)(x-b)}$$

$$\begin{aligned}
 1. \int \frac{dx}{\sqrt{(x-a)(b-x)}} &= 2 \arcsin \sqrt{\frac{x-a}{b-x}} + C (a < b) \\
 2. \int \sqrt{(x-a)(b-x)} dx &= \frac{2x-a-b}{4} \sqrt{(x-a)(b-x)} + \frac{(b-a)^2}{4} \arcsin \sqrt{\frac{x-a}{b-x}} + C, (a < b)
 \end{aligned}$$

三角函数的积分

$$\begin{aligned}
 1. \int \tan x dx &= -\ln |\cos x| + C \\
 2. \int \cot x dx &= \ln |\sin x| + C \\
 3. \int \sec x dx &= \ln \left| \tan \left(\frac{\pi}{2} + \frac{x}{2} \right) \right| + C = \ln |\sec x + \tan x| + C \\
 4. \int \csc x dx &= \ln \left| \tan \frac{x}{2} \right| + C = \ln |\csc x - \cot x| + C \\
 5. \int \sec^2 x dx &= \tan x + C \\
 6. \int \csc^2 x dx &= -\cot x + C
 \end{aligned}$$

$$\begin{aligned}
 7. \int \sec x \tan x dx &= \sec x + C \\
 8. \int \csc x \cot x dx &= -\csc x + C \\
 9. \int \sin^2 x dx &= \frac{x}{2} - \frac{1}{4} \sin 2x + C \\
 10. \int \cos^2 x dx &= \frac{x}{2} + \frac{1}{4} \sin 2x + C \\
 11. \int \sin^n x dx &= -\frac{1}{n} \sin^{n-1} x \cos x + \frac{n-1}{n} \int \sin^{n-2} x dx \\
 12. \int \cos^n x dx &= \frac{1}{n} \cos^{n-1} x \sin x + \frac{n-1}{n} \int \cos^{n-2} x dx \\
 13. \int \frac{dx}{\sin^n x} &= -\frac{1}{n-1} \frac{\cos x}{\sin^{n-1} x} + \frac{n-2}{n-1} \int \frac{dx}{\sin^{n-2} x} \\
 14. \int \frac{dx}{\cos^n x} &= \frac{1}{n-1} \frac{\sin x}{\cos^{n-1} x} + \frac{n-2}{n-1} \int \frac{dx}{\cos^{n-2} x} \\
 15. &
 \end{aligned}$$

$$\begin{aligned}
 &\int \cos^m x \sin^n x dx \\
 &= \frac{1}{m+n} \cos^{m-1} x \sin^{n+1} x + \frac{m-1}{m+n} \int \cos^{m-2} x \sin^n x dx \\
 &= -\frac{1}{m+n} \cos^{m+1} x \sin^{n-1} x + \frac{n-1}{m+1} \int \cos^m x \sin^{n-2} x dx
 \end{aligned}$$

$$\begin{aligned}
 16. \int \frac{dx}{a+b \sin x} &= \begin{cases} \frac{2}{\sqrt{a^2-b^2}} \arctan \frac{a \tan \frac{x}{2} + b}{\sqrt{a^2-b^2}} + C & (a^2 > b^2) \\ \frac{1}{\sqrt{b^2-a^2}} \ln \left| \frac{a \tan \frac{x}{2} + b - \sqrt{b^2-a^2}}{a \tan \frac{x}{2} + b + \sqrt{b^2-a^2}} \right| + C & (a^2 < b^2) \end{cases} \\
 17. \int \frac{dx}{a+b \cos x} &= \begin{cases} \frac{2}{a+b} \sqrt{\frac{a+b}{a-b}} \arctan \left(\sqrt{\frac{a-b}{a+b}} \tan \frac{x}{2} \right) + C & (a^2 > b^2) \\ \frac{1}{a+b} \sqrt{\frac{a+b}{a-b}} \ln \left| \frac{\tan \frac{x}{2} + \sqrt{\frac{a+b}{b-a}}}{\tan \frac{x}{2} - \sqrt{\frac{a+b}{b-a}}} \right| + C & (a^2 < b^2) \end{cases}
 \end{aligned}$$

$$\begin{aligned}
 18. \int \frac{dx}{a^2 \cos^2 x + b^2 \sin^2 x} &= \frac{1}{ab} \arctan \left(\frac{b}{a} \tan x \right) + C \\
 19. \int \frac{dx}{a^2 \cos^2 x - b^2 \sin^2 x} &= \frac{1}{2ab} \ln \left| \frac{b \tan x + a}{b \tan x - a} \right| + C \\
 20. \int x \sin ax dx &= \frac{1}{a^2} \sin ax - \frac{1}{a} x \cos ax + C \\
 21. \int x^2 \sin ax dx &= -\frac{1}{a} x^2 \cos ax + \frac{2}{a^2} x \sin ax + \frac{2}{a^3} \cos ax + C \\
 22. \int x \cos ax dx &= \frac{1}{a^2} \cos ax + \frac{1}{a} x \sin ax + C \\
 23. \int x^2 \cos ax dx &= \frac{1}{a} x^2 \sin ax + \frac{2}{a^2} x \cos ax - \frac{2}{a^3} \sin ax + C
 \end{aligned}$$

反三角函数的积分 (其中 $a > 0$)

$$\begin{aligned}
 1. \int \arcsin \frac{x}{a} dx &= x \arcsin \frac{x}{a} + \sqrt{a^2 - x^2} + C \\
 2. \int x \arcsin \frac{x}{a} dx &= \left(\frac{x^2}{2} - \frac{a^2}{4} \right) \arcsin \frac{x}{a} + \frac{x}{4} \sqrt{x^2 - x^2} + C \\
 3. \int x^2 \arcsin \frac{x}{a} dx &= \frac{x^3}{3} \arcsin \frac{x}{a} + \frac{1}{9} (x^2 + 2a^2) \sqrt{a^2 - x^2} + C \\
 4. \int \arccos \frac{x}{a} dx &= x \arccos \frac{x}{a} - \sqrt{a^2 - x^2} + C \\
 5. \int x \arccos \frac{x}{a} dx &= \left(\frac{x^2}{2} - \frac{a^2}{4} \right) \arccos \frac{x}{a} - \frac{x}{4} \sqrt{a^2 - x^2} + C
 \end{aligned}$$

$$\begin{aligned}
 6. \int x^2 \arccos \frac{x}{a} dx &= \frac{x^3}{3} \arccos \frac{x}{a} - \frac{1}{9} (x^2 + 2a^2) \sqrt{a^2 - x^2} + C \\
 7. \int \arctan \frac{x}{a} dx &= x \arctan \frac{x}{a} - \frac{a}{2} \ln(a^2 + x^2) + C \\
 8. \int x \arctan \frac{x}{a} dx &= \frac{1}{2} (a^2 + x^2) \arctan \frac{x}{a} - \frac{a}{2} x + C \\
 9. \int x^2 \arctan \frac{x}{a} dx &= \frac{x^3}{3} \arctan \frac{x}{a} - \frac{a}{6} x^2 + \frac{a^3}{6} \ln(a^2 + x^2) + C
 \end{aligned}$$

指数函数的积分

$$\begin{aligned}
 1. \int a^x dx &= \frac{1}{\ln a} a^x + C \\
 2. \int e^{ax} dx &= \frac{1}{a} e^{ax} + C \\
 3. \int x e^{ax} dx &= \frac{1}{a^2} (ax - 1) e^{ax} + C \\
 4. \int x^n e^{ax} dx &= \frac{1}{a} x^n e^{ax} - \frac{n}{a} \int x^{n-1} e^{ax} dx \\
 5. \int x a^x dx &= \frac{x}{\ln a} a^x - \frac{1}{(\ln a)^2} a^x + C \\
 6. \int x^n a^x dx &= \frac{1}{\ln a} x^n a^x - \frac{n}{\ln a} \int x^{n-1} a^x dx \\
 7. \int e^{ax} \sin bx dx &= \frac{1}{a^2 + b^2} e^{ax} (a \sin bx - b \cos bx) + C \\
 8. \int e^{ax} \cos bx dx &= \frac{1}{a^2 + b^2} e^{ax} (b \sin bx + a \cos bx) + C \\
 9. \int e^{ax} \sin^n bx dx &= \frac{1}{a^2 + b^2 n^2} e^{ax} \sin^{n-1} bx (a \sin bx - nb \cos bx) + \frac{n(n-1)b^2}{a^2 + b^2 n^2} \int e^{ax} \sin^{n-2} bx dx \\
 10. \int e^{ax} \cos^n bx dx &= \frac{1}{a^2 + b^2 n^2} e^{ax} \cos^{n-1} bx (a \cos bx + nb \sin bx) + \frac{n(n-1)b^2}{a^2 + b^2 n^2} \int e^{ax} \cos^{n-2} bx dx
 \end{aligned}$$

对数函数的积分

$$\begin{aligned}
 1. \int \ln x dx &= x \ln x - x + C \\
 2. \int \frac{dx}{x \ln x} &= \ln |\ln x| + C \\
 3. \int x^n \ln x dx &= \frac{1}{n+1} x^{n+1} (\ln x - \frac{1}{n+1}) + C \\
 4. \int (\ln x)^n dx &= x (\ln x)^n - n \int (\ln x)^{n-1} dx \\
 5. \int x^m (\ln x)^n dx &= \frac{1}{m+1} x^{m+1} (\ln x)^n - \frac{n}{m+1} \int x^m (\ln x)^{n-1} dx
 \end{aligned}$$

STL 积分/求和 (need std::)

$$\begin{aligned}
 1. \int_0^1 t^{x-1} (1-t)^{y-1} dt &= \operatorname{beta}(x, y) = \frac{\Gamma(x)\Gamma(y)}{\Gamma(x+y)} \\
 2. \int_0^{+\infty} t^{num-1} e^{-t} dt &= \operatorname{tgamma}(num) = e^{\operatorname{lgamma}(num)} = \Gamma(num) \\
 3. \int_{num}^{+\infty} \frac{e^{-t}}{t} dt &= -\operatorname{expint}(-num) \\
 4. \sum_{n=1}^{+\infty} n^{-num} &= \operatorname{riemann_zeta}(num) \\
 5. \frac{2}{\sqrt{\pi}} \int_0^{arg} e^{-t^2} dt &= \operatorname{erf}(arg) \\
 6. \int_0^{phi} \frac{d\theta}{\sqrt{1-k^2 \sin^2 \theta}} &= \operatorname{ellint_1}(k, phi) \\
 7. \int_0^{phi} \sqrt{1-k^2 \sin^2 \theta} d\theta &= \operatorname{ellint_2}(k, phi) \\
 &\text{椭圆离心率: } e = \sqrt{1 - \frac{b^2}{a^2}}, \text{ 周长: } 2a \operatorname{ellint_2}(e, \pi)
 \end{aligned}$$